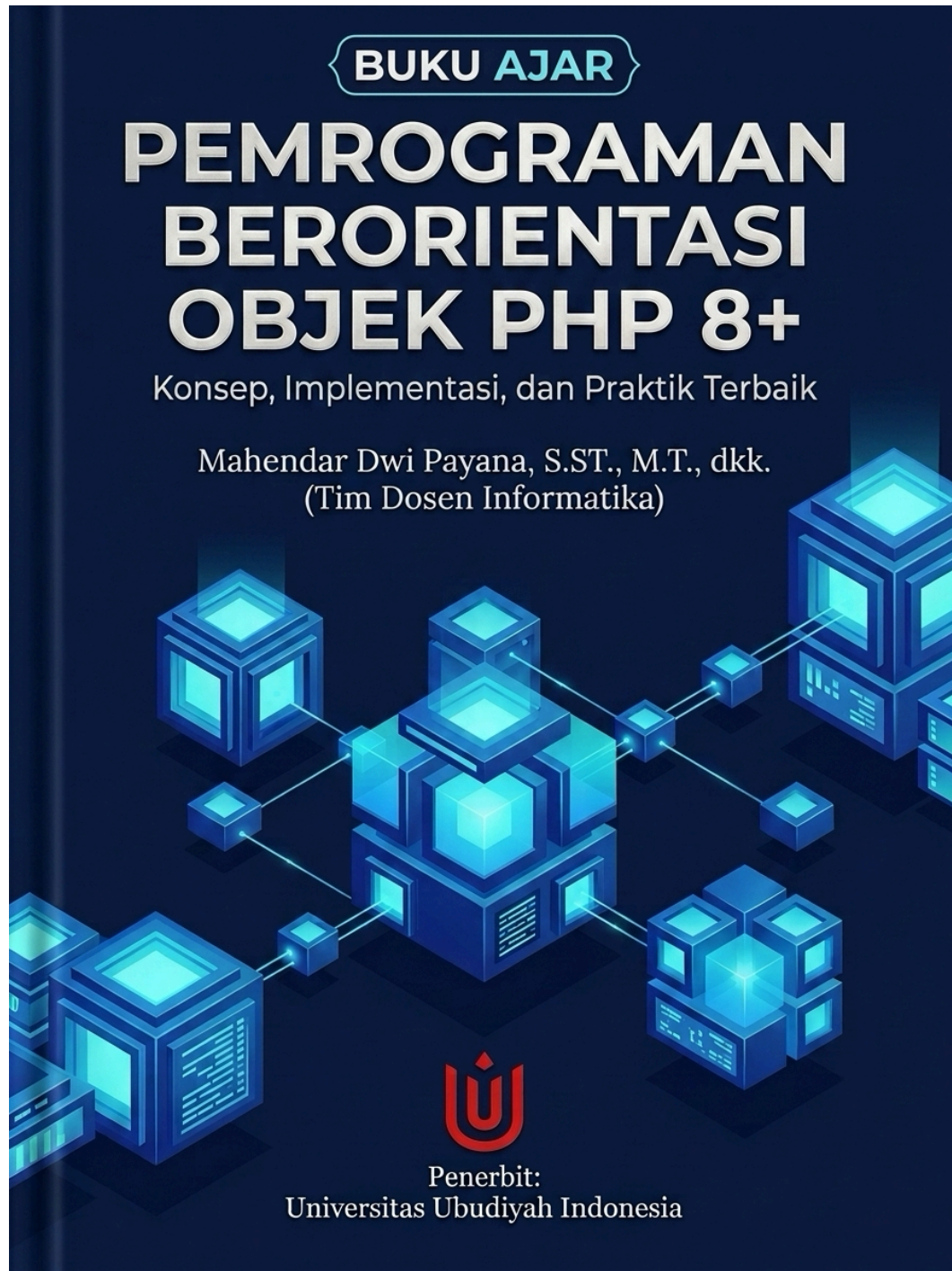




**BUKU AJAR PERKULIAHAN**

# **PEMROGRAMAN BERORIENTASI OBJEK MENGUNAKAN PHP 8+**

*Pendekatan Teoretis, Praktik Rekayasa Perangkat Lunak Modern,  
dan Implementasi Arsitektur Bersih*

---

Tim Penulis:

**Mahendar Dwi Payana, S.ST., M.T.**

**Desita Ria Yusian TB, S.ST., M.T**

**M.Bayu Wibawa, S.Kom., MMSI**

**Nurul Hamdi, S.T., M.Kom**

**Zuhar Musliyana, S.ST., M.T**

**PROGRAM STUDI INFORMATIKA  
FAKULTAS SAINS DAN TEKNOLOGI  
UNIVERSITAS UBUDİYAH INDONESIA  
BANDA ACEH  
2026**

## INFORMASI PENERBITAN & HAK CIPTA

|                                |                                                                                                                                                                                                       |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Judul Buku</b>              | Buku Ajar Pemrograman Berorientasi Objek menggunakan PHP 8+                                                                                                                                           |
| <b>Penulis</b>                 | 1. Mahendar Dwi Payana, S.ST., M.T.<br>2. Desita Ria Yusian TB, S.ST., M.T<br>3. M.Bayu Wibawa, S.Kom., MMSI<br>4. Nurul Hamdi, S.T., M.Kom<br>5. Zuhar Musliyana, S.ST., M.T                         |
| <b>Editor Ahli</b>             | Tim Pengembang Kurikulum Program Studi Informatika UUI                                                                                                                                                |
| <b>Desain &amp; Tata Letak</b> | Laboratorium Rekayasa Perangkat Lunak & Sistem Informasi UUI                                                                                                                                          |
| <b>Penerbit</b>                | UUI Press / Program Studi Informatika<br>Fakultas Sains dan Teknologi, Universitas Ubudiyah Indonesia                                                                                                 |
| <b>Alamat Redaksi</b>          | Jl. Alue Naga, Desa Tibang, Kec. Syiah Kuala, Kota Banda Aceh, Aceh<br>23114<br>Laman: <a href="https://uui.ac.id">https://uui.ac.id</a>   Pos-el: <a href="mailto:info@uui.ac.id">info@uui.ac.id</a> |
| <b>Edisi &amp; Cetakan</b>     | Cetakan Pertama, 2025 (Versi Rilis: v1.0.0)                                                                                                                                                           |
| <b>Nomor ISBN</b>              | 978-623-XXXX-XX-X (e-Book Digital Reference)                                                                                                                                                          |

### Katalog Dalam Terbitan (KDT)

PAYANA, Mahendar Dwi, dkk.

Buku Ajar Pemrograman Berorientasi Objek menggunakan PHP 8+ /  
Mahendar Dwi Payana, Desita Ria Yusian TB, M.Bayu Wibawa,  
Nurul Hamdi, Zuhar Musliyana ; editor, Tim Reviewer Informatika UUI.  
-- Cet. 1 -- Banda Aceh : UUI Press, 2025.  
xvi, 175 hlm. : illus. ; 29,7 cm. (Format A4)

Bibliografi : hlm. 162-164  
Glosarium : hlm. 158-161  
ISBN 978-623-XXXX-XX-X

1. Pemrograman Berorientasi Objek (Ilmu Komputer)    2. PHP 8+  
I. Judul    II. Tim Reviewer Informatika UUI

005.133 -- DDC23

### PERINGATAN HAK CIPTA & SANKSI PELANGGARAN:

Hak Cipta © 2025 pada Penulis. Hak Penerbitan pada UUI Press. Dilarang memperbanyak atau memindahkan sebagian atau seluruh isi buku ini dalam bentuk apa pun, baik secara elektronik maupun mekanik, termasuk memfotokopi, merekam, atau dengan sistem penyimpanan lainnya tanpa izin tertulis dari Penulis dan Penerbit. Sanksi Pelanggaran Pasal 113 UU No. 28 Tahun 2014: Setiap orang yang dengan tanpa hak melakukan pelanggaran hak ekonomi dapat dipidana dengan pidana penjara paling lama 10 (sepuluh) tahun dan/atau denda paling banyak Rp 4.000.000.000,00 (empat miliar rupiah).

## KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah SWT, karena atas limpahan rahmat, taufik, dan hidayah-Nya, Buku Ajar berjudul "Pemrograman Berorientasi Objek menggunakan PHP 8+" ini dapat diselesaikan dengan baik. Buku ini disusun secara khusus sebagai buku referensi utama dan pegangan perkuliahan bagi mahasiswa Program Studi Informatika, Fakultas Sains dan Teknologi, Universitas Ubudiyah Indonesia, maupun para pembelajar mandiri yang ingin menguasai rekayasa perangkat lunak modern berbasis PHP.

Perkembangan teknologi perangkat lunak saat ini menuntut setiap pengembang untuk menulis kode program yang tidak sekadar bekerja, melainkan juga harus modular, terstruktur, mudah diuji (*testable*), serta siap dikembangkan dalam skala besar (*scalable*). Paradigma Pemrograman Berorientasi Objek (*Object-Oriented Programming/OOP*) telah menjadi landasan fundamental dalam industri perangkat lunak dunia. Bersamaan dengan itu, bahasa pemrograman PHP telah mengalami evolusi revolusioner, khususnya sejak rilis PHP versi 8.0, 8.1, 8.2, dan seterusnya. Fitur modern seperti *Constructor Property Promotion*, *Type Safety* yang ketat, *Readonly Properties*, *Backed Enums*, serta ekosistem manajemen pustaka Composer dengan standar PSR-4 menjadikan PHP sebagai bahasa kelas industri yang sangat tangguh.

Buku ajar ini disusun secara berjenjang (*pedagogical ladder*) yang memandu pembaca mulai dari filosofi dasar pergeseran paradigma prosedural menuju berorientasi objek, penguasaan empat pilar utama OOP (Enkapsulasi, Pewarisan, *Polimorfisme*, dan Abstraksi), penanganan kesalahan tangguh (*Exception Handling*), manipulasi koleksi data dan file I/O, hingga penerapan prinsip desain SOLID dan arsitektur Model-Service-Repository dalam bentuk proyek terintegrasi.

Penulis menyadari bahwa buku ini masih memiliki ruang untuk perbaikan dan penyempurnaan di masa mendatang. Oleh karena itu, saran, masukan konstruktif, dan kritik yang membangun dari para sejawat akademisi, praktisi industri, serta mahasiswa senantiasa penulis nantikan.

Akhir kata, penulis menyampaikan apresiasi yang mendalam kepada pimpinan Universitas Ubudiyah Indonesia, rekan-rekan dosen di lingkungan Fakultas Sains dan Teknologi, serta para mahasiswa yang senantiasa memberikan inspirasi selama proses belajar mengajar. Semoga buku ajar ini dapat memberikan manfaat nyata, menginspirasi, dan menjadi bekal kompetensi yang kokoh bagi kemajuan keilmuan teknologi informasi di tanah air.

Banda Aceh, 2025

**Tim Penulis**  
**Universitas Ubudiyah Indonesia**

Hlm. 4

## CAPAIAN PEMBELAJARAN MATA KULIAH

Mata kuliah Pemrograman Berorientasi Objek (Kode MK: IFR 214, Bobot: 3 SKS) dirancang untuk membekali mahasiswa dengan kompetensi berikut:

| Kode CPL | Deskripsi Capaian Pembelajaran Lulusan (CPL)                                                                           |
|----------|------------------------------------------------------------------------------------------------------------------------|
| CPL-P1   | Menguasai konsep, paradigma, prinsip, dan teknik pemrograman berorientasi objek dalam rekayasa perangkat lunak.        |
| CPL-KU1  | Mampu berpikir logis, sistematis, kritis, dan solutif dalam merancang abstraksi perangkat lunak.                       |
| CPL-KK1  | Mampu merancang, mengimplementasikan, menguji, dan mendokumentasikan aplikasi berorientasi objek dengan PHP 8+.        |
| CPL-S1   | Menunjukkan sikap profesional, berintegritas, mandiri, dan beretika akademik dalam penyelesaian tugas perangkat lunak. |

### Daftar Capaian Pembelajaran Mata Kuliah (CPMK)

| Kode CPMK | Deskripsi Capaian Pembelajaran Mata Kuliah                                                                   |
|-----------|--------------------------------------------------------------------------------------------------------------|
| CPMK-1    | Mampu membandingkan paradigma prosedural vs berorientasi objek dan memahami ekosistem PHP 8+.                |
| CPMK-2    | Mampu mengimplementasikan class, objek, constructor promotion, method, dan struktur memori.                  |
| CPMK-3    | Mampu menerapkan 4 pilar OOP: Encapsulation, Inheritance, Polymorphism, dan Abstraction.                     |
| CPMK-4    | Mampu mengelola arsitektur proyek melalui Namespace, Autoloading PSR-4, Exception Handling, dan Collections. |
| CPMK-5    | Mampu menganalisis dan menerapkan prinsip desain perangkat lunak SOLID pada aplikasi.                        |
| CPMK-6    | Mampu merancang dan membangun proyek aplikasi OOP PHP terintegrasi berbasis studi kasus riil.                |

## DAFTAR ISI

### ■ BAGIAN AWAL

|                                           |     |
|-------------------------------------------|-----|
| Halaman Judul & Informasi Penulis         | i   |
| Informasi Penerbitan & Hak Cipta (KDT)    | ii  |
| Kata Pengantar Penulis                    | iii |
| Matriks Capaian Pembelajaran (CPL & CPMK) | iv  |
| Daftar Isi Lengkap                        | v   |

### ■ BATANG TUBUH PEMBELAJARAN

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>BAB 1: PENGANTAR PARADIGMA OOP &amp; EKOSISTEM PHP 8+</b>         | <b>1</b>  |
| 1.1 Hakikat dan Evolusi Paradigma Pemrograman                        | 1         |
| 1.2 Perbandingan Komprehensif: Prosedural vs Berorientasi Objek      | 3         |
| 1.3 Empat Pilar Fundamental Pemrograman Berorientasi Objek           | 6         |
| 1.4 Transformasi Modern: Mengapa PHP 8+ Menjadi Standar Baru?        | 8         |
| <b>BAB 2: ANATOMI CLASS, OBJEK, DAN MANAJEMEN MEMORI</b>             | <b>12</b> |
| 2.1 Konsep Entitas Dunia Nyata dan Pemodelan Class                   | 12        |
| 2.2 Deklarasi Class dan Typed Properties di PHP 8+                   | 14        |
| 2.3 Instansiasi Objek dan Operator Arrow (->)                        | 16        |
| 2.4 Variabel Pseudo \$this dan Lingkup Eksekusi                      | 18        |
| 2.5 Pengelolaan Memori: Object Handle vs Kloning (clone)             | 20        |
| <b>BAB 3: METHOD, CONSTRUCTOR PROMOTION, DAN SIKLUS HIDUP OBJEK</b>  | <b>24</b> |
| 3.1 Siklus Hidup Objek dan Inisialisasi Otomatis                     | 24        |
| 3.2 Pergeseran Paradigma: Constructor Property Promotion             | 26        |
| 3.3 Named Arguments pada PHP 8.0+                                    | 28        |
| 3.4 Member Statis dan Pola Static Factory Method                     | 31        |
| <b>BAB 4: ENKAPSULASI, VISIBILITY MODIFIERS, DAN READONLY</b>        | <b>36</b> |
| 4.1 Filosofi Enkapsulasi: Information Hiding & State Invariant       | 36        |
| 4.2 Tiga Tingkat Hak Akses & Prinsip Tell, Don't Ask                 | 38        |
| 4.3 Praktik Terbaik: Mutasi Terkendali vs Public Properties          | 40        |
| 4.4 Modern Immutability: readonly Properties & Class                 | 42        |
| 4.5 PHP 8.4+: Asymmetric Visibility & Property Hooks                 | 44        |
| <b>BAB 5: PEWARISAN (INHERITANCE) DAN KOMPOSISI TRAIT</b>            | <b>48</b> |
| 5.1 Filosofi Pewarisan: Taksonomi Is-A & Bahaya Fragile Base Class   | 48        |
| 5.2 Sintaks Pewarisan, Keyword extends, dan Penempatan parent::      | 50        |
| 5.3 Mengunci Perilaku: final Class, Method, dan Constants (PHP 8.1+) | 53        |
| 5.4 Trait: Horizontal Code Reuse & Resolusi Konflik (insteadof / as) | 55        |
| <b>BAB 6: POLIMORFISME (POLYMORPHISM) DAN DYNAMIC DISPATCH</b>       | <b>60</b> |
| 6.1 Hakikat Polimorfisme: Klasifikasi Cardelli & Wegner (1985)       | 60        |
| 6.2 Dynamic Method Dispatch pada Runtime Engine                      | 62        |
| 6.3 Polymorphic Type Hinting & Polymorphic Collections               | 64        |

|                                                                                 |            |
|---------------------------------------------------------------------------------|------------|
| 6.4 Operator instanceof & Type Narrowing                                        | 67         |
| 6.5 Studi Kasus Enterprise: Multi-Channel Notification Engine                   | 69         |
| <b>BAB 7: ABSTRAKSI: INTERFACE, ABSTRACT CLASS, DAN BACKED ENUM</b>             | <b>72</b>  |
| 7.1 Filosofi Abstraksi: Pemisahan Kontrak (What) dan Implementasi (How)         | 72         |
| 7.2 Abstract Class & Template Method Pattern                                    | 74         |
| 7.3 Interface: Kontrak Murni & Multiple Interface Implementation                | 76         |
| 7.4 Matriks Komparasi Akademis: Abstract Class vs Interface                     | 79         |
| 7.5 Integrasi Backed Enum di PHP 8.1+ & Pattern Matching                        | 81         |
| <b>BAB 8: MANAJEMEN NAMESPACE, STANDAR PSR-4, DAN COMPOSER</b>                  | <b>84</b>  |
| 8.1 Filosofi Namespace & Masalah Global Namespace Pollution                     | 84         |
| 8.2 Resolusi Simbol Namespace: FQN, Qualified, dan Unqualified                  | 86         |
| 8.3 Standar PSR-4 Autoloading & Struktur Direktori Composer                     | 88         |
| 8.4 Konfigurasi composer.json & Optimasi Autoloading Produksi                   | 91         |
| 8.5 Studi Kasus Arsitektur Clean Namespace PSR-4                                | 93         |
| <b>BAB 9: PENANGANAN KESALAHAN (EXCEPTION HANDLING) &amp; ERROR FLOW</b>        | <b>96</b>  |
| 9.1 Filosofi Exception Handling & Hierarki Throwable                            | 96         |
| 9.2 Blok Kontrol try-catch-finally & Multi-Catch Syntax                         | 98         |
| 9.3 Fitur Modern PHP 8+: throw Expression & Non-Capturing Catches               | 101        |
| 9.4 Merancang Custom Domain Exceptions & Exception Chaining (\$previous)        | 103        |
| 9.5 Studi Kasus Transaksi Perbankan Terpadu                                     | 105        |
| <b>BAB 10: KOLEKSI OBJEK DAN MANIPULASI ARRAY MODERN</b>                        | <b>108</b> |
| 10.1 Filosofi First-Class Collections & Masalah Type Safety                     | 108        |
| 10.2 Mengintegrasikan Antarmuka Standar PHP (SPL: Countable, IteratorAggregate) | 110        |
| 10.3 Manipulasi Fungsional: map, filter, reduce, dan Arrow Functions            | 113        |
| 10.4 Studi Kasus Analisis Rekapitulasi Yudisium                                 | 116        |
| <b>BAB 11: MANAJEMEN BERKAS DAN ALIRAN DATA (FILE I/O STREAM)</b>               | <b>120</b> |
| 11.1 Filosofi Persistensi Berkas & Pencegahan Race Condition (flock LOCK_EX)    | 120        |
| 11.2 Pemrosesan Berkas Berorientasi Objek dengan SplFileObject                  | 122        |
| 11.3 Manipulasi JSON Modern & Validasi Native di PHP 8.3+ (json_validate)       | 125        |
| 11.4 Pola Desain: File-Based Repository Pattern                                 | 127        |
| <b>BAB 12: PRINSIP DESAIN PERANGKAT LUNAK SOLID PADA PHP MODERN</b>             | <b>132</b> |
| 12.1 Fondasi Teoretis SOLID: Menghindari Pembusukan Perangkat Lunak             | 132        |
| 12.2 Bedah Komprehensif 5 Prinsip SOLID di PHP 8+ (SRP, OCP, LSP, ISP, DIP)     | 134        |
| 12.3 Pipeline Checkout E-Commerce Berstandar SOLID                              | 139        |
| <b>BAB 13: ARSITEKTUR APLIKASI (MODEL-SERVICE-REPOSITORY)</b>                   | <b>144</b> |
| 13.1 Filosofi Clean Architecture & Arsitektur Berlapis (Dependency Rule)        | 144        |
| 13.2 Lapisan 1: Entity / Domain Model dengan Backed Enum                        | 146        |
| 13.3 Repository Interface (Kontrak Abstraksi di Lapisan Domain)                 | 148        |
| 13.4 Service Layer (Use Case / Business Orchestrator)                           | 150        |
| 13.5 Implementasi Repository Konkret (Lapisan Infrastructure)                   | 152        |
| <b>BAB 14: STUDI KASUS MINI PROJECT: POINT OF SALE (POS) TERPADU</b>            | <b>156</b> |
| 14.1 Deskripsi Kasus & Kebutuhan Sistem POS Enterprise                          | 156        |
| 14.2 Desain Domain Model & First-Class Collection                               | 158        |
| 14.3 Strategi Diskon & Pembayaran Polimorfik (SOLID OCP & DIP)                  | 161        |

|                                                               |     |
|---------------------------------------------------------------|-----|
| 14.4 Application Service & Persistensi File-Based Terproteksi | 163 |
| 14.5 Panduan Evaluasi & Rubrik Penilaian Capstone Project     | 165 |
| <b>■ BAGIAN AKHIR</b>                                         |     |
| Glosarium Istilah Rekayasa Perangkat Lunak & OOP PHP 8+       | 168 |
| Daftar Pustaka & Referensi Akademis Terindeks                 | 172 |
| Profil Penulis & Tim Reviewer                                 | 175 |



## BAB 1

# PENGANTAR PARADIGMA PEMROGRAMAN BERORIENTASI OBJEK & EKOSISTEM PHP 8+

---

### Tujuan & Capaian Pembelajaran (Sub-CPMK 1)

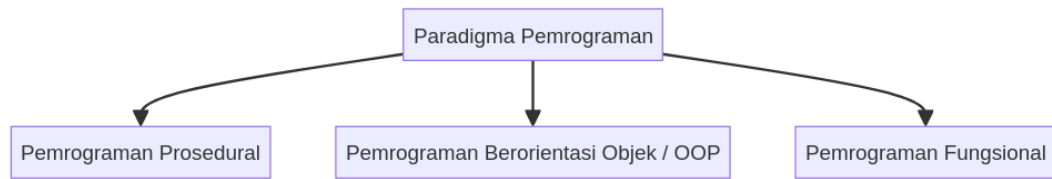
- Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:
  - Menjelaskan konsep fundamental paradigma pemrograman dan pergeserannya dari prosedural ke berorientasi objek.
- Membedakan secara analitis antara struktur program prosedural dan struktur modular berbasis objek.
- Mengidentifikasi fitur-fitur revolusioner pada PHP modern (PHP 8.0, 8.1, 8.2, 8.3).
- Menyiapkan lingkungan pengembangan perangkat lunak berbasis PHP 8+ dan Composer.

### 1.1 Hakikat dan Evolusi Paradigma Pemrograman

Dalam ilmu komputasi dan rekayasa perangkat lunak, paradigma pemrograman (*programming paradigm*) merupakan kerangka konseptual, model berpikir, dan metodologi fundamental yang mendikte bagaimana seorang pengembang menstrukturkan data, mengeksekusi logika, dan menyelesaikan persoalan komputasi. Sebagaimana halnya paradigma dalam sains, cara pandang ini menentukan alat bantu, sintaksis, serta batas-batas arsitektur yang digunakan dalam membangun program komputer.

Sepanjang sejarah perkembangan rekayasa perangkat lunak, terdapat tiga paradigma utama yang paling dominan mempengaruhi cara perangkat lunak dibangun di industri:

- **Pemrograman Prosedural (*Procedural Programming*):** Pendekatan berbasis instruksi sekuensial dan pemanggilan fungsi/prosedur yang memanipulasi data global atau lokal secara terpisah.
- **Pemrograman Berorientasi Objek (*Object-Oriented Programming - OOP*):** Pendekatan yang memandang sistem sebagai interaksi antar entitas mandiri (objek) yang membungkus status data (properti) dan perilakunya (*method*) ke dalam satu kesatuan utuh.
- **Pemrograman Fungsional (*Functional Programming*):** Pendekatan deklaratif yang memandang komputasi sebagai evaluasi fungsi matematis murni tanpa efek samping (*side effects*) dan tanpa status yang berubah-ubah (*immutable state*).



Gambar 1. Paradigma Pemrograman

## 1.2 Perbandingan Komprehensif: Prosedural vs Berorientasi Objek

Pada masa-masa awal bahasa PHP (era PHP 3 dan PHP 4), gaya penulisan kode didominasi oleh pendekatan prosedural. Pengembang menuliskan serangkaian berkas PHP yang berisi gabungan kode logika, instruksi SQL, manipulasi *string*, dan *tag* HTML dalam satu berkas tunggal yang linier (sering diistilahkan sebagai '*spaghetti code*').

Pendekatan prosedural memiliki kelebihan berupa kesederhanaan untuk skrip kecil (*quick and dirty scripts*). Namun, ketika skala aplikasi berkembang menjadi ribuan baris kode dengan puluhan modul bisnis, pendekatan prosedural mengalami masalah serius: data global mudah terkontaminasi, fungsi saling bergantung secara ketat (*tightly coupled*), dan pengujian unit (unit testing) menjadi hampir mustahil dilakukan.

### Pendekatan Prosedural di PHP:

```

<?php
// Kode Prosedural: Data dan Fungsi Terpisah
function hitungLuasPersegiPanjang(float $panjang, float $lebar): float {
    return $panjang * $lebar;
}

$panjang = 12.5;
$lebar = 4.0;
$luas = hitungLuasPersegiPanjang($panjang, $lebar);
echo "Luas Bangun: " . $luas . " cm²";
  
```

### Pendekatan Berorientasi Objek di PHP 8+:

```

<?php
declare(strict_types=1);

// Kode OOP: Data ($panjang, $lebar) dan Perilaku (hitungLuas) Menyatu
class PersegiPanjang {
    public function __construct(
        private float $panjang,
        private float $lebar
    ) {
        if ($panjang <= 0 || $lebar <= 0) {
            throw new InvalidArgumentException("Dimensi harus bernilai positif.");
        }
    }

    public function hitungLuas(): float {
        return $this->panjang * $this->lebar;
    }
}
  
```

```

    }
}

$bangun = new PersegiPanjang(12.5, 4.0);
echo "Luas Bangun: " . $bangun->hitungLuas() . " cm2";

```

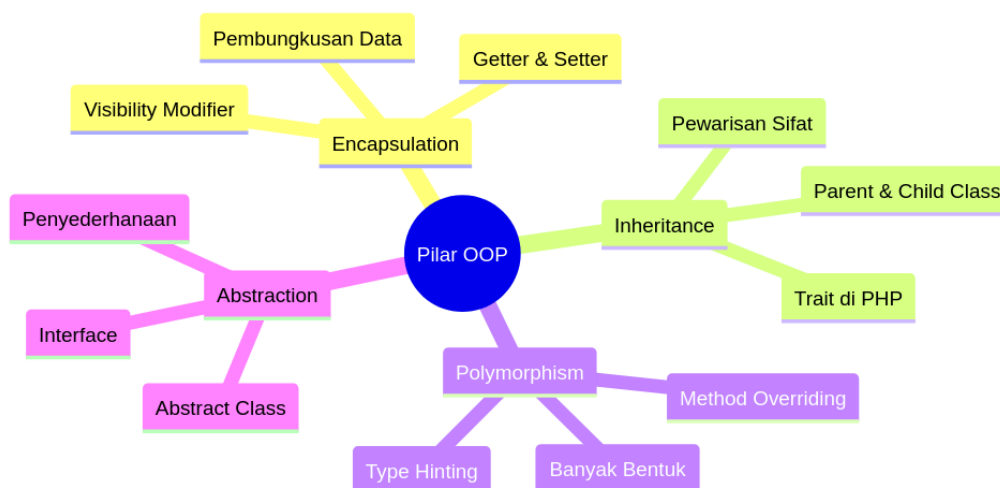
| Dimensi Analisis         | Pemrograman Prosedural                                       | Pemrograman Berorientasi Objek (OOP)                                                           |
|--------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Satuan Dasar Organisasi  | Fungsi, <i>Subroutine</i> , dan Prosedur                     | <i>Class</i> dan <i>Instance</i> Objek                                                         |
| Relasi Data & Perilaku   | Terpisah; data mengalir bebas ke berbagai fungsi             | Tersatukan secara kohesif di dalam entitas Objek                                               |
| Keamanan Status Data     | Rendah; variabel global rentan diubah tanpa validasi         | Tinggi; dilindungi melalui Enkapsulasi & Access <i>Modifiers</i>                               |
| <i>Reusabilitas</i> Kode | Rendah hingga sedang (mengandalkan <i>copy-paste</i> fungsi) | Sangat tinggi (menggunakan <i>Inheritance</i> , <i>Trait</i> , & <i>Polimorfisme</i> )         |
| Kemudahan Pemeliharaan   | Menurun drastis seiring bertambahnya kompleksitas            | Tinggi; modul terisolasi memudahkan <i>debugging</i> & <i>refactoring</i>                      |
| Penerapan Industri       | Skrip otomatisasi singkat, pemrosesan data mikro             | <i>Framework Enterprise</i> ( <i>Laravel</i> , <i>Symfony</i> , <i>Yii2</i> , <i>Laminas</i> ) |

### 1.3 Empat Pilar Fundamental Pemrograman Berorientasi Objek

Arsitektur OOP bertumpu kokoh di atas empat pilar utama. Penguasaan mendalam atas keempat pilar ini merupakan syarat mutlak bagi setiap rekayasawan perangkat lunak:

- 1. Enkapsulasi (*Encapsulation*):** Mekanisme penggabungan data (*state*) dan metode pemroses data (*behavior*) ke dalam satu wadah tunggal (*class*), seraya menyembunyikan detail internal yang sensitif dari akses luar langsung (*Information Hiding*).
- 2. Pewarisan (*Inheritance*):** Kemampuan sebuah *class* baru (*subclass/child*) untuk mewarisi atribut, metode, dan sifat dari *class* yang sudah ada (*superclass/parent*), mendorong efisiensi penulisan kode sesuai prinsip DRY (*Don't Repeat Yourself*).

3. **Polimorfisme (*Polymorphism*)**: Prinsip 'satu antarmuka, banyak rupa', yang memungkinkan objek dari berbagai class turunan berbeda merespons pemanggilan pesan atau *method* yang sama dengan perilaku spesifik masing-masing melalui *dynamic dispatch*.
4. **Abstraksi (*Abstraction*)**: Proses penyederhanaan kompleksitas sistem dengan hanya menampilkan antarmuka penting dan relevan kepada pengguna luar, seraya menyembunyikan mekanisme teknis internal yang rumit.



Gambar 2. Empat Pilar OOP dan Turunannya

## 1.4 Transformasi Modern: Mengapa PHP 8+ Menjadi Standar Baru?

PHP (PHP: *Hypertext Preprocessor*) telah bertransformasi secara radikal. PHP 8.x bukan lagi sekadar bahasa skrip pelengkap HTML, melainkan bahasa pemrograman berorientasi objek yang lengkap, tangguh, memiliki sistem tipe data statis (*Strict Typing*), serta dilengkapi JIT (*Just-In-Time*) compiler.

| Fitur Modern                          | Rilis   | Signifikansi & Dampak Arsitektural                                               |
|---------------------------------------|---------|----------------------------------------------------------------------------------|
| <i>Typed Properties</i>               | PHP 7.4 | Menjamin integritas tipe data properti secara statis sejak kompilasi.            |
| <i>Constructor Property Promotion</i> | PHP 8.0 | Mengeliminasi <i>boilerplate</i> kode inisialisasi class hingga 70%.             |
| <i>Named Arguments</i>                | PHP 8.0 | Memungkinkan pemanggilan parameter fungsi secara eksplisit tanpa terikat urutan. |

|                            |         |                                                                                        |
|----------------------------|---------|----------------------------------------------------------------------------------------|
| <i>Union Types (A/B)</i>   | PHP 8.0 | Mendukung deklarasi <i>multi-tipe</i> parameter secara legal dan <i>type-safe</i> .    |
| <i>Readonly Properties</i> | PHP 8.1 | Menjamin kekekalan nilai ( <i>immutability</i> ) properti setelah instansiasi.         |
| <i>Backed Enums</i>        | PHP 8.1 | Tipe data enumerasi <i>native</i> dengan nilai skalar untuk validasi status yang aman. |
| <i>Readonly Classes</i>    | PHP 8.2 | Menjadikan seluruh properti <i>class</i> secara otomatis bersifat <i>readonly</i> .    |



#### Tips Praktik Industri: Disiplin Deklarasi Strict Types

Selalu biasakan menyematkan deklarasi `'declare(strict_types=1);'` pada baris pertama setiap berkas program PHP Anda. Deklarasi ini memaksa Zend Engine menolak konversi tipe data otomatis (*type coercion*) yang acapkali menjadi celah *bug* kritis dan inkonsistensi data di lingkungan produksi *enterprise*.

## Rangkuman Materi

1. Paradigma OOP mengorganisasikan sistem komputasi ke dalam entitas-entitas objek yang menggabungkan status data dan perilaku.
2. Empat pilar utama OOP meliputi Enkapsulasi, Pewarisan, *Polimorfisme*, dan Abstraksi.
3. PHP 8+ membawa perubahan masif dengan menghadirkan sistem tipe yang ketat, *constructor promotion*, *readonly properties*, dan performa tinggi.
4. Pengembangan aplikasi skala besar wajib memanfaatkan standar pengorganisasian modern untuk menjaga skalabilitas dan kemudahan uji.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan kelemahan mendasar pemrograman prosedural jika diterapkan pada sistem informasi rumah sakit berskala besar!
2. Uraikan bagaimana prinsip *Information Hiding* dalam enkapsulasi melindungi integritas data finansial perbankan!
3. Sebutkan dan jelaskan 3 fitur baru PHP 8.x yang secara signifikan meningkatkan produktivitas penulisan kode berorientasi objek!
4. Instal PHP 8.1+ di komputer kerja Anda, periksa versinya melalui terminal (``php -v``), dan buatlah skrip pengujian sederhana bertipe data ketat!

## BAB 2

# ANATOMI CLASS, OBJEK, DAN MANAJEMEN MEMORI

### Tujuan & Capaian Pembelajaran (Sub-CPMK 2)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Mendefinisikan perbedaan ontologis antara *Class* sebagai *blueprint* dan Objek sebagai instansi fisik di memori.
- Mendeklarasikan *Class* dengan *Typed Properties* dan *Method* fungsional sesuai standar PSR-12.
- Melakukan instansiasi objek dan mengakses properti serta *method* menggunakan operator *arrow* (`->`).
- Menjelaskan peran variabel pseudo `$this` dalam konteks *runtime* objek.
- Menganalisis pengelolaan referensi memori (*Object Handle*) dan mekanisme kloning objek (`clone`).

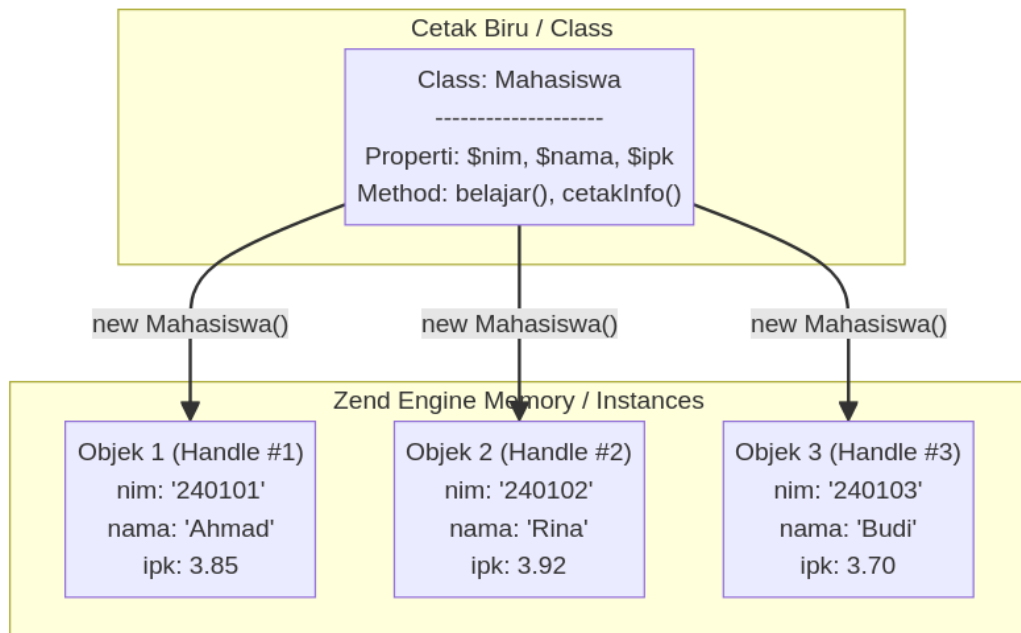
## 2.1 Konsep Entitas di Dunia Nyata dan Pemodelan Class

Dalam rekayasa perangkat lunak berorientasi objek, dunia nyata dimodelkan sebagai kumpulan objek. **Objek** nyata memiliki dua karakteristik dasar: karakteristik deskriptif (atribut/ciri-ciri) dan karakteristik operasional (tindakan/perilaku). Sebagai contoh, seorang Mahasiswa memiliki atribut berupa Nomor Induk Mahasiswa (NIM), Nama Lengkap, Program Studi, dan Indeks Prestasi Kumulatif (IPK). Di samping itu, mahasiswa memiliki perilaku seperti belajar, mengambil mata kuliah, dan membayar biaya kuliah.

**Class** adalah cetak biru (*blueprint*), *deskriptor*, atau skema rancangan abstrak yang mendefinisikan atribut apa saja yang akan dimiliki serta perilaku apa saja yang dapat dilakukan oleh suatu kelompok entitas. Sedangkan **Objek** adalah perwujudan konkret (*instance*) dari cetak biru tersebut yang dialokasikan di dalam memori komputer dan memiliki nilai data uniknya sendiri. Berikut contoh penerapan *class* dan *object* pada dunia nyata:

- **Class:** Struktur, cetak biru, atau tipe data abstrak bentukan pengembang yang mendefinisikan atribut dan perilaku.
- **Object:** Wujud konkret yang dialokasikan di memori (*Zend Engine Memory*) dan memiliki data spesifik (*state*).

Di atas adalah contoh penerapan *class* dan *object* dimana contoh kasus penerapan tersebut dapat dilihat pada Gambar 3 dibawah ini.



Gambar 3. Konsep Fundamental Class dan Object

## 2.2 Deklarasi Class dan Typed Properties di PHP 8+

Sejak era PHP 7.4 dan disempurnakan di PHP 8+, PHP mewajibkan penulisan tipe data pada properti class (Typed Properties). Praktik ini sangat krusial guna mencegah bug akibat ketidaksesuaian tipe data pada saat runtime.

```
<?php
declare(strict_types=1);

namespace App\Model;

class Mahasiswa
{
    public string $nim;
    public string $nama;
    public string $programStudi = "Informatika";
    public float $ipk = 0.0;
    public bool $isActive = true;

    public function perbaruiIpk(float $ipkBaru): void
    {
        if ($ipkBaru < 0.0 || $ipkBaru > 4.0) {
            throw new \InvalidArgumentException("Rentang IPK harus antara 0.00 hingga 4.00!");
        }
        $this->ipk = $ipkBaru;
    }

    public function cetakKarakteristik(): void
    {
        $status = $this->isActive ? "Aktif" : "Cut/Non-Aktif";
        echo "=== KARTU TANDA MAHASISWA ===\n";
        echo "NIM          : {$this->nim}\n";
    }
}
```

```

        echo "Nama Lengkap   : {$this->nama}\n";
        echo "Program Studi  : {$this->programStudi}\n";
        echo "IPK Terkini    : " . number_format($this->ipk, 2) . "\n";
        echo "Status         : {$status}\n";
        echo "=====\n";
    }
}

```

## 2.3 Instansiasi Objek dan Operator Arrow (`->`)

Untuk menciptakan objek fisik dari suatu class, bahasa PHP menggunakan kata kunci `new`. Ketika `new Mahasiswa()` dieksekusi, *Zend Engine* (mesin eksekusi inti PHP) akan mengalokasikan satu blok memori baru, menginisialisasi tabel simbol properti, dan mengembalikan penunjuk (*handle*) objek tersebut kepada variabel penampung.

```

<?php
require_once 'Mahasiswa.php';
use App\Model\Mahasiswa;

$mhs1 = new Mahasiswa();
$mhs1->nim = "240101001";
$mhs1->nama = "Cut Meurah Intan";
$mhs1->perbaruiIpk(3.88);

$mhs2 = new Mahasiswa();
$mhs2->nim = "240101002";
$mhs2->nama = "Teuku Rayhan";
$mhs2->perbaruiIpk(3.72);

$mhs1->cetakKarakteristik();
$mhs2->cetakKarakteristik();

```

## 2.4 Variabel Pseudo `$this` dan Lingkup Eksekusi

Dalam Pemrograman Berbasis Objek (*Object-Oriented Programming / OOP*), sebuah *class* bertindak sebagai cetak biru (*blueprint*), sedangkan *object* adalah wujud nyata (*instance*) yang dibuat dari cetak biru tersebut di memori. Ketika sebuah *method* dipanggil, sistem perlu mengetahui **objek mana** yang sedang memanggil dan memproses data dalam *method* tersebut. Untuk itulah variabel pseudo **`$this`** digunakan.

Variabel pseudo `$this` secara otomatis menunjuk ke instansi objek yang saat itu sedang mengeksekusi *method* bersangkutan (*current object instance*). Melalui `$this->namaProperti`, kode di dalam *method* dapat membaca dan memodifikasi status internal objek tersebut secara akurat.

### 2.4.1 Pengertian dan Definisi Variabel Pseudo `$this`



Variabel **\$this** dinamakan **variabel pseudo** (variabel semu) karena variabel ini tidak dideklarasikan secara manual oleh pemrogram, melainkan dibuat dan dikelola secara otomatis oleh *engine* bahasa pemrograman (seperti PHP).

- **Fungsi Utama:** **\$this** menyimpan referensi (*reference*) ke objek yang **saat itu sedang aktif** menjalankan *method* bersangkutan (*current object instance*).
- **Akses Properti dan Method:** Dengan operator panah (**->**), **\$this** digunakan untuk mengakses properti dan *method* internal milik objek itu sendiri dari dalam lingkup (*scope*) *method*.

**Aturan Penting:** Variabel **\$this** hanya ada dan hanya valid di dalam *instance method* (method non-statis). **\$this** tidak dapat digunakan di dalam *static method* maupun di luar lingkup kelas (*global scope*).

## 2.4.2 Mengapa \$this Sangat Diperlukan? (Masalah Ambiguitas Lingkup)

Sering kali, nama parameter yang diterima oleh *method* sama persis dengan nama properti yang dimiliki oleh *class*. Variabel **\$this** memecahkan masalah **ambiguitas variabel** antara variabel lokal/parameter dengan properti objek.

### Contoh Tanpa Ketersediaan Context (Konseptual):

Jika sebuah parameter bernama **\$nama** dan properti kelas juga bernama **\$nama**, bahasa pemrograman membutuhkan cara untuk membedakan keduanya:

```
class Pengguna {
    public $nama; // Properti Objek

    public function setName($nama) { // $nama adalah Parameter Lokal
        // Tanpa $this, compiler/interpreter bingung:
        // Apakah variabel Lokal diisi variabel Lokal sendiri?
        $nama = $nama;
    }
}
```

### Solusi Menggunakan \$this:

Dengan **\$this->nama**, kita secara eksplisit memberitahu sistem: "*Isilah properti **nama** milik objek ini dengan nilai dari parameter **\$nama**.*"

```
class Pengguna {
    public $nama;

    public function setName($nama) {
```

```

    $this->nama = $nama; // $this->nama mengarah ke properti objek
}
}

```

## 2.5 Pengelolaan Memori di PHP: Object Handle vs Kloning (`clone`)

Pernahkah Anda membagikan *link* Google Docs ke teman Anda untuk diedit bersama? Saat teman Anda mengubah judul dokumen lewat *link* tersebut, dokumen di akun Anda ikut berubah. Mengapa? Karena Anda berdua memegang **akses ke file yang sama**, bukan memiliki dua file yang berbeda. Prinsip yang persis sama berlaku pada pengelolaan memori objek di PHP.

### 2.5.1 Memahami Object Handle (Si "Kunci Lemari")

Banyak pengembang pemula mengira bahwa variabel objek menyimpan seluruh data fisik objek tersebut. Padahal, variabel penampung objek di PHP hanya menyimpan sebuah **Object Identifier (Handle)**—bisa kita bayangkan sebagai **kunci** menuju lokasi penyimpanan di memori.

Ketika kita melakukan *assignment* biasa ( $\$b = \$a$ ), PHP **tidak menggandakan objeknya**, melainkan hanya membuat **kunci duplikat** yang mengarah ke lokasi memori yang sama.

```

<?php
$a = new Mahasiswa();
$a->nama = "Budi";

$b = $a;
$b->nama = "Siti";
echo $a->nama; // Output: "Siti"

$c = clone $a;
$c->nama = "Andi";
echo $a->nama; // Output: Tetap "Siti"
echo $c->nama; // Output: "Andi"

```

#### Apa yang terjadi di balik layar?

1.  $\$a = \text{new Mahasiswa}()$ ; → Objek fisik dibuat di memori, lalu kuncinya diberikan ke  $\$a$ .
2.  $\$b = \$a$ ; → PHP memberikan Kunci Duplikat ke  $\$b$ . Sekarang  $\$a$  dan  $\$b$  memegang kunci ke rumah yang sama.
3. Ketika  $\$b->nama = \text{"Siti"};$  dijalankan, isi "rumah" diubah. Maka saat dipanggil via  $\$a$ , namanya sudah berubah menjadi "Siti".

### 2.5.2 Kloning Objek (clone): Membuat Objek Baru yang Terpisah

Lalu bagaimana jika kita benar-benar ingin menggandakan objek beserta seluruh isinya tanpa mengganggu objek yang asli? Di sinilah kita menggunakan kata kunci **clone**. Kata kunci **clone** memerintahkan PHP untuk membaca data pada objek lama, membuat lokasi memori baru (objek fisik baru), lalu meng-copy datanya secara mandiri.

```
<?php
// Mengkloning Objek (Membuat fisik objek baru di memori)
$c = clone $a;
$c->nama = "Andi";

echo $a->nama; // Output: Tetap "Siti"
echo $c->nama; // Output: "Andi"
```

Apa yang terjadi di balik layar?

1. **\$c = clone \$a;** → PHP membangun "rumah baru" yang bentuk dan isinya identik dengan rumah **\$a**, lalu menyerahkan kunci rumah baru tersebut ke variabel **\$c**.
2. Ketika **\$c->nama = "Andi";** diubah, yang berubah hanya rumah milik **\$c**.
3. Rumah lama milik **\$a** (yang juga diakses oleh **\$b**) tetap aman dan tidak terpengaruh sama sekali (nilainya tetap **"Siti"**).

### 2.5.3 Ringkasan Konseptual

- **Assignment (\$b = \$a)** → Seperti berbagi *link* atau membuat kunci duplikat. Hanya ada **1 Objek**, tetapi diakses oleh **2 Variabel**. Perubahan di satu tempat akan berdampak ke tempat lain.
- **Kloning (\$c = clone \$a)** → Seperti *copy-paste* file. Menghasilkan **2 Objek terpisah** di memori. Perubahan pada objek kloningan tidak akan mempengaruhi objek aslinya.

#### **Tips Efisiensi Memori: Pahami Karakteristik Object Handle**

Ingatlah bahwa penugasan **'\$b = \$a'** di PHP hanya menyalin Object Handle (penunjuk memori), bukan menggandakan fisik objek. Manfaatkan sifat ini untuk menghemat alokasi RAM ketika melewati entitas besar ke service layer. Gunakan kata kunci 'clone' hanya ketika Anda benar-benar memerlukan duplikasi fisik independen.

### Rangkuman Materi

- *Class* bertindak sebagai *template* atau cetak biru abstrak, sementara *Object* adalah wujud fisik yang memakan memori komputer.
- Typed properties menjamin ketepatan tipe data dan mencegah bug runtime yang tidak terduga.

- Variabel pseudo ``$this`` merujuk ke instance objek yang sedang aktif mengeksekusi method.
- Variabel objek di PHP bekerja berdasarkan Object Handle. Untuk menduplikasi objek fisik secara terpisah, gunakan kata kunci ``clone``.

### Soal Latihan & Penugasan Mandiri

1. Buatlah class ``BukuPerpustakaan`` dengan properti ``$isbn`` (string), ``$judul`` (string), ``$penulis`` (string), ``$stok`` (int), dan method ``pinjamBuku(int $jumlah): bool``!
2. Jelaskan apa yang terjadi di memori komputer ketika instruksi ``$obj2 = $obj1`` dijalankan dibandingkan dengan ``$obj2 = clone $obj1``!
3. Mengapa PHP melempar pesan error ``Typed property must not be accessed before initialization`` dan bagaimana cara mencegahnya?
4. Tuliskan sebuah skrip untuk mendemonstrasikan perilaku dua objek dari class yang sama namun memiliki state data yang berbeda!

## BAB 3

# METHOD, CONSTRUCTOR PROPERTY PROMOTION, DAN SIKLUS HIDUP OBJEK

---

### Tujuan & Capaian Pembelajaran (Sub-CPMK 2)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami siklus hidup objek mulai dari alokasi awal, inisialisasi constructor, hingga pelepasan oleh destructor.
- Menerapkan magic method `\_\_construct()` dan `\_\_destruct()` secara tepat.
- Menguasai fitur modern PHP 8.0: Constructor Property Promotion untuk efisiensi kode.
- Memanfaatkan Named Arguments dan Union Types pada pemanggilan method.
- Membedakan fungsi member statis (`static`) dengan member instansiasi, serta menerapkan pola Static Factory Method.

## 3.1 Pendahuluan & Peran Method dalam Objek

Jika **properti** mewakili *apa yang dimiliki* oleh sebuah objek (data/status), maka **method** mewakili *apa yang dapat dilakukan* oleh objek tersebut (perilaku/aksi). Method pada dasarnya adalah fungsi yang dideklarasikan di dalam sebuah kelas dan memiliki akses khusus ke status internal objek tempat ia bernaung.

Dalam pengembangan perangkat lunak modern, penulisan method bukan sekadar membuat deretan perintah, melainkan merancang kontrak perilaku yang aman, jelas, dan mudah dipahami oleh pengembang lain.

### 3.1.1 Anatomi Deklarasi Method Modern

Sejak hadirnya PHP 7 dan PHP 8, penulisan method makin disiplin dengan adanya Type Hinting (penjelas tipe data) dan Return Type Declaration (deklarasi tipe kembalian).

```
<?php
declare(strict_types=1);

class RekeningTabungan {
    private float $saldo;

    public function __construct(float $saldoAwal) {
        $this->saldo = $saldoAwal;
    }
}
```

```
// Method dengan Parameter Type Hinting dan Return Type Declaration
public function setor(float $jumlah): float {
    if ($jumlah <= 0) {
        throw new InvalidArgumentException("Jumlah setoran harus lebih dari nol.");
    }

    $this->saldo += $jumlah;
    return $this->saldo;
}

public function getSaldo(): float {
    return $this->saldo;
}
}
```

### 3.1.2 Manfaat Tipe Data Ketat (Strict Typing)

Dengan menambahkan `declare(strict_types=1);` dan menentukan tipe data parameter/kembalian:

- **Mencegah Bug Tersembunyi:** Sistem akan menolak jika kita memasukkan string yang tidak valid ke dalam parameter `float`.
- **Dokumentasi Mandiri:** Kode menjadi self-documenting; pengembang lain tidak perlu menebak tipe data apa yang diharapkan oleh sebuah method.

## 3.2 Constructor & Destructor: Awal dan Akhir Hidup Objek

Setiap objek memiliki perjalanan hidup: ia diciptakan, menjalankan tugasnya di memori, dan akhirnya dihapus ketika tidak lagi dibutuhkan. PHP menyediakan dua *magic method* utama untuk mengelola momen awal dan akhir ini: `__construct()` dan `__destruct()`.

### 3.2.1 Constructor (`__construct()`): Momen Kelahiran Objek

Constructor adalah method khusus yang dieksekusi secara otomatis oleh sistem persis saat objek dibuat menggunakan kata kunci `new`.

- **Fungsi Utama:** Melakukan inisialisasi awal, seperti menetapkan nilai dasar properti, membuka koneksi basis data, atau menyiapkan dependensi yang dibutuhkan objek.

```
<?php
declare(strict_types=1);

class PengirimEmail {
    private string $serverSmtp;
    private int $port;
```

```
// Constructor dipanggil saat 'new PengirimEmail()'
public function __construct(string $serverSmtp, int $port = 587) {
    $this->serverSmtp = $serverSmtp;
    $this->port = $port;
    echo "Koneksi ke server serverSmtp:port berhasil disiapkan.\n";
}
}

$pengirim = new PengirimEmail("smtp.gmail.com");
```

### 3.2.2 Destructor (\_\_destruct): Momen Perpisahan Objek

Destructor adalah kebalikan dari constructor. Method `__destruct()` akan dipanggil **secara otomatis** ketika objek hendak dihapus dari memori—baik karena skrip PHP selesai dieksekusi, variabelnya di`unset()`, atau objek keluar dari lingkungannya (*scope*).

- **Fungsi Utama:** Melakukan pembersihan (*cleanup*), seperti menutup koneksi database, menghapus berkas sementara (*temporary file*), atau menutup kunci sesi (*session lock*).

```
<?php
class KoneksiDatabase {
    private string $namaTabel;

    public function __construct(string $namaTabel) {
        $this->namaTabel = $namaTabel;
        echo "[CONNECT] Berhasil terhubung ke tabel {$this->namaTabel}.\n";
    }

    public function query(string $sql): void {
        echo "[EXECUTE] Menjalankan query: $sql\n";
    }

    public function __destruct() {
        echo "[DISCONNECT] Memutus koneksi dari tabel
{$this->namaTabel}.\n";
    }
}

// Simulasi Lingkup Eksekusi
function jalankanTransaksi() {
    $db = new KoneksiDatabase("users");
    $db->query("SELECT * FROM users");
    // Saat fungsi selesai, $db keluar scope dan destructor dipanggil
    otomatis
}
```

```

jalankanTransaksi();
echo "Skrip selesai.\n";

```

Output:

```

[CONNECT] Berhasil terhubung ke tabel users.
[EXECUTE] Menjalankan query: SELECT * FROM users
[DISCONNECT] Memutus koneksi dari tabel users.
Skrip selesai.

```

### 3.3 Constructor Property Promotion (PHP 8+)

Sebelum PHP 8.0, penulisan **class** sederhana yang memiliki banyak properti memerlukan banyak kode perulangan (*boilerplate code*). Kita harus mendefinisikan properti, meneruma variabel di *constructor*, lalu melakukan *assignment* satu per satu via **\$this**.

#### 3.3.1 Pendekatan Tradisional (PHP 7.x dan sebelumnya)

```

class Produk {
    private string $id;
    private string $nama;
    private float $harga;
    private int $stok;

    public function __construct(string $id, string $nama, float $harga, int
$stok) {
        $this->id = $id;
        $this->nama = $nama;
        $this->harga = $harga;
        $this->stok = $stok;
    }
}

```

Tampak melelahkan dan penuh pengulangan kata.

#### 3.3.2. Pendekatan Modern dengan Constructor Property Promotion (PHP 8+)

Dengan **Constructor Property Promotion**, kita dapat menggabungkan deklarasi properti, visibilitas (**private**, **protected**, **public**), dan penetapan nilai langsung dalam parameter **\_\_construct()**.

```

class Produk {
    // Properti otomatis dideklarasikan dan diisi nilainya sekaligus!
    public function __construct(
        private string $id,
        private string $nama,
        private float $harga,
        private int $stok = 0
    ) {}
}

```



```

}

// Penggunaan tetap sama persis:
$laptop = new Produk("P-001", "Laptop Gaming", 15000000.0, 5);

```

### 3.3.3. Kombinasi dengan Properti Imut (*Readonly*)

Di PHP 8.1+, kita bahkan bisa menambahkan penentu **readonly**. Artinya, properti diinisialisasi melalui constructor dan **tidak dapat diubah nilainya lagi** sepanjang hidup objek tersebut.

```

class UserDTO {
    public function __construct(
        public readonly string $uuid,
        public readonly string $email
    ) {}
}

$user = new UserDTO("123-abc", "user@example.com");
// $user->email = "baru@example.com"; // Error! Properti readonly tidak dapat diubah

```

```

<?php
class Pengguna {
    public function __construct(
        public string $username,
        public string $email,
        public string $role = "Mahasiswa",
        public bool $isVerifikasi = false,
        public string $zonaWaktu = "Asia/Jakarta"
    ) {}
}

$user = new Pengguna(
    email: "mahendar@uui.ac.id",
    username: "mahendar",
    zonaWaktu: "Asia/Banda_Aceh"
);

```

## 3.4 Siklus Hidup Objek (*Object Lifecycle*) dan Memory Cleanup

Untuk menguasai OOP secara menyeluruh, seorang pengembang perlu memahami bagaimana objek berkelana di dalam memori internal komputer dari lahir hingga tiada.

**Tabel 3.1 Siklus Hidup Objek**

| Tahapan              | Deskripsi/Aksi         |
|----------------------|------------------------|
| 1. Instansiasi (new) | Alokasi memori di Heap |

|                            |                                                 |
|----------------------------|-------------------------------------------------|
| 2. Inisialisasi            | Executing <code>__construct()</code>            |
| 3. Penggunaan/Aktif        | Membaca/mengubah state via <code>\$this</code>  |
| 4. Pelepasan Referensi     | Nilai variabel di-unset / keluar scope          |
| 5. Garbage Collection (GC) | Executing <code>__destruct()</code> & Dealokasi |

### 3.4.1. Tahap-Tahap Siklus Hidup Objek

1. **Instansiasi (*new*)**: PHP mengalokasikan ruang memori di area **Heap** dan membuat *Object Handle* unik.
2. **Inisialisasi**: `__construct()` dipanggil untuk mengisi data dasar objek.
3. **Masa Aktif**: Objek berinteraksi dengan komponen lain dalam aplikasi.
4. **Kehilangan Referensi**: Ketika tidak ada lagi variabel yang memegang *handle* objek tersebut (misalnya variabel di-`unset()`, dialihkan ke objek lain, atau fungsi pemanggil telah selesai).
5. **Garbage Collection & Destruction**: *Garbage Collector* (GC) PHP mendeteksi objek yang tak tersentuh ini, menjalankan `__destruct()`, lalu melepaskan ruang memori Heap kembali ke sistem operasi.

```
<?php
class Logger {
    public function __construct(private string $file) {
        echo "Membuka file log: {$this->file}\n";
    }

    public function __destruct() {
        echo "Menutup file log: {$this->file}\n";
    }
}

// Kasus A: Pelepasan Otomatis (Keluar Scope)
function testScope() {
    $log1 = new Logger("app.log");
} // $log1 dihancurkan otomatis di sini saat fungsi selesai

testScope();

// Kasus B: Pelepasan Manual via unset()
log2=newLogger("error.log");unset(log2); // __destruct() langsung dipanggil saat ini juga

echo "Akhir dari program.\n";
```

### 3.5 Magic Methods Populer Terkait Objek

Selain `__construct()` dan `__destruct()`, PHP menyediakan metode spesial berawalan dua garis bawah (`__`) yang membantu objek merespons skenario tertentu secara anggun:

#### 3.5.1. `__toString()`

Metode ini otomatis dipanggil ketika sebuah objek dicoba untuk dicetak sebagai *string* (misalnya menggunakan `echo` atau `print`).

```
<?php
class Person {
    public function __construct(
        private string $nama,
        private int $umur
    ) {}

    public function __toString(): string {
        return "Person [Nama: {$this->nama}, Umur: {$this->umur} tahun]";
    }
}

$budi = new Person("Budi Santoso", 25);
echo $budi; // Output: Person [Nama: Budi Santoso, Umur: 25 tahun]
```

#### 3.5.2. `__invoke()`

Metode ini memungkinkan sebuah objek diperlakukan dan dipanggil seolah-olah ia adalah sebuah fungsi (*callable*).

```
class Diskontroler {
    public function __invoke(float $hargaOriginal, float $diskon): float {
        return $hargaOriginal - ($hargaOriginal * $diskon);
    }
}

$hitungDiskon = new Diskontroler();
// Dipanggil langsung seperti fungsi!
$hargaAkhir = $hitungDiskon(100000, 0.2);
echo "Harga setelah diskon: Rp" . $hargaAkhir; // Output: Rp80000
```



#### **Tips Rekayasa: Kombinasi Constructor Promotion & Readonly**

Gabungkan modifier 'public readonly' langsung di dalam parameter constructor PHP 8+. Pola ini menghasilkan Data Transfer Object (DTO) dan Value Object yang sangat ringkas, aman dari mutasi liar (immutable), serta secara otomatis bersifat self-documenting.

## Rangkuman Materi

1. *Constructor* (`__construct`) dieksekusi secara otomatis saat instansiasi untuk menginisialisasi status objek.
2. *Constructor Property Promotion* di PHP 8 menyederhanakan deklarasi properti dan parameter menjadi satu kesatuan ringkas.
3. *Named Arguments* memungkinkan pengisian parameter secara eksplisit, acak urutan, dan melewati nilai default.
4. Member statis dapat diakses langsung melalui *Class* (`self::` atau `ClassName::`), dan *Static Factory Method* adalah standar industri untuk instansiasi objek multi-varian.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan keuntungan arsitektural penggunaan *Constructor Property Promotion* dibandingkan cara deklarasi klasik!
2. Bagaimana peran destructor (`__destruct()`) dalam pengelolaan koneksi jaringan atau berkas?
3. Rancanglah sebuah class `MataKuliah` dengan properti `$kodeMK`, `$namaMK`, `$sks`, dan `$dosenPengampu` menggunakan PHP 8 *Constructor Promotion* dan *Named Arguments*!
4. Implementasikan *Static Factory Method* pada class `TransaksiKasir` untuk membuat transaksi tipe 'Tunai', 'Debit', dan 'QRIS'!

## BAB 4

# ENKAPSULASI, VISIBILITY MODIFIERS, READONLY, DAN ASYMMETRIC VISIBILITY

### Tujuan & Capaian Pembelajaran (Sub-CPMK 3)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami filosofi fundamental Enkapsulasi, *Information Hiding*, dan konsep *State Invariant*.
- Menguasai secara mendalam 3 tingkatan *Visibility Modifiers*: *public*, *protected*, dan *private* pada properti, *method*, dan konstanta.
- Mengimplementasikan Getter (Accessor) dan Setter (Mutator) dengan validasi aturan domain bisnis serta pola Method Chaining.
- Menerapkan fitur modern readonly Properties (PHP 8.1+) dan readonly class (PHP 8.2+) untuk menjamin kekekalan status data (Data Immutability).
- Menganalisis dan mengimplementasikan fitur mutakhir PHP 8.4: Asymmetric Visibility (*public private(set)*) dan Property Hooks (*get/set*).
- Memahami mekanisme kendali properti dinamis melalui magic methods (*\_\_get*, *\_\_set*, *\_\_isset*, *\_\_unset*) serta aturan penolakan Dynamic Properties di PHP 8.2+.

## 4.1 Filosofi dan Fondasi Teoretis Enkapsulasi

Dalam rekayasa perangkat lunak modern, Enkapsulasi (*Encapsulation*) merupakan pilar fundamental pertama yang menyatukan data (*state/property*) dan perilaku pemroses data (*behavior/method*) ke dalam satu unit struktural mandiri yang kohesif (*Class*), seraya menyembunyikan detail representasi internal yang sensitif dari akses luar langsung (*Information Hiding*).

Prinsip *Information Hiding* pertama kali dicetuskan secara ilmiah oleh David Parnas pada tahun 1972 dalam karya klasiknya mengenai dekomposisi modular sistem perangkat lunak. Parnas menegaskan bahwa modul perangkat lunak yang bermutu tinggi harus menyembunyikan keputusan perancangan internalnya dari modul lain. Kode pemanggil (*client code*) cukup berinteraksi dengan antarmuka publik (*public interface*) yang stabil tanpa perlu mengetahui bagaimana struktur data fisik disimpan atau dimanipulasi di dalam memori.

Tujuan utama enkapsulasi adalah memelihara *State Invariant*, yaitu kondisi atau aturan kebenaran bisnis yang harus selalu terjamin validitasnya di setiap saat sepanjang masa hidup objek. Sebagai contoh, saldo rekening perbankan tidak boleh bernilai negatif secara sembarangan, usia pasien rumah sakit harus berada pada rentang biologis logis (0 hingga 130 tahun), dan koordinat GPS harus berada pada rentang batas bola bumi (-90 s.d. +90 untuk latitude).

Jika seluruh properti dibiarkan *public*, pihak luar dapat merusak *invariant* kapan saja (`$rekening->saldo = -999999;`). Melalui enkapsulasi, akses penulisan dikunci rapat dan hanya dapat dilakukan melalui *method* yang memeriksa *invariant* terlebih dahulu.

### Prinsip 'Tell, Don't Ask' vs Antipattern Anemic Domain Model:

Salah satu jebakan yang sering dialami oleh pemula adalah menerapkan enkapsulasi setengah hati dengan membuat semua properti *private*, namun langsung membuat *method getter* dan *setter* untuk setiap properti tanpa logika bisnis apa pun. Pendekatan ini menghasilkan *Anemic Domain Model* (objek yang hanya menjadi kantong data pasif tanpa kecerdasan bisnis).

Prinsip desain berorientasi objek yang elegan adalah Tell, Don't Ask (Perintahkan, Jangan Bertanya): Alih-alih kode luar mengambil saldo lalu menghitung sendiri pemotongan uang dan mengeset nilai baru, kode luar cukup memerintahkan objek rekening untuk mengeksekusi penarikan uang (`$rekening->tarikTunai($nominal, $pin);`). Objek rekening secara mandiri akan memvalidasi otorisasi PIN, memastikan kecukupan saldo, mengurangi saldo internal, dan mencatat riwayat mutasi audit.

## 4.2 Tiga Tingkat Hak Akses (Visibility Modifiers) di PHP

PHP menyediakan tiga kata kunci pengatur hak akses (*Visibility Modifiers*) yang dapat disematkan pada *property*, *method*, dan konstanta *class*:

| Modifier  | Akses dari Dalam Class | Akses dari Child Class (extends) | Akses dari Luar (Client Code) | Tingkat Keamanan          |
|-----------|------------------------|----------------------------------|-------------------------------|---------------------------|
| public    | ✓ Diizinkan penuh      | ✓ Diizinkan penuh                | ✓ Bebas diakses langsung      | Terbuka (Public API)      |
| protected | ✓ Diizinkan penuh      | ✓ Diizinkan penuh (Inheritance)  | ✗ Dilarang (Fatal Error)      | Internal Keluarga Warisan |
| private   | ✓ Diizinkan penuh      | ✗ Dilarang (Hanya class asal)    | ✗ Dilarang (Terkunci rapat)   | Terkunci Paling Rapat     |

```
<?php
declare(strict_types=1);

class RekeningInduk
{
    public string $nomorRekening;    // Bebas dibaca dan ditulis publik
    protected float $saldo = 0.0;    // Hanya bisa diakses class ini dan subclass
    private string $pinRahasia;      // Terkunci rapat hanya untuk class RekeningInduk
}
```

```

    public function __construct(string $nomor, float $saldoAwal, string $pin) {
        $this->nomorRekening = $nomor;
        $this->saldo          = $saldoAwal;
        $this->pinRahasia     = $pin;
    }

    protected function verifikasiPin(string $pinInput): bool {
        return $this->pinRahasia === $pinInput;
    }
}

class RekeningTabungan extends RekeningInduk
{
    public function ambilUang(float $nominal, string $pinInput): void {
        // ✓ Boleh mengakses method protected milik parent:
        if (!$this->verifikasiPin($pinInput)) {
            throw new \DomainException("Autentikasi PIN gagal!");
        }
        if ($nominal > $this->saldo) {
            throw new \UnderflowException("Saldo tabungan tidak mencukupi!");
        }
        $this->saldo -= $nominal; // ✓ Boleh memodifikasi properti protected parent
    }
}

```

#### Hak Akses pada Konstanta Class (Class Constants):

Sejak era PHP 7.1+, konstanta di dalam *class* dapat dikunci menggunakan *visibility modifier* untuk mencegah kebocoran konfigurasi internal ke pihak luar:

```

<?php
class KonfigurasiKeamanan {
    public const VERSI_SISTEM = "3.2.1";           // Konfigurasi publik
    protected const BATAS_LOGIN_GAGAL = 5;        // Hanya untuk subclass autentikasi
    private const KUNCI_ENKRIPSI = "UUI-KEY-2025"; // Rahasia internal mutlak
}

```

### 4.3 Getter, Setter, dan Validasi Aturan Domain Bisnis

*Getter* (*Accessor*) bertugas membaca nilai properti *private* secara terkontrol, sedangkan *Setter* (*Mutator*) bertugas memodifikasi nilai properti seraya menegakkan aturan validasi bisnis (*Business Rules Enforcement*).

Penerapan *Method Chaining* (*Fluent Interface*) pada *setter* dilakukan dengan mengembalikan referensi objek saat ini (`return $this;`), sehingga pengembang dapat memperbarui beberapa nilai properti dalam satu rangkaian instruksi yang ringkas dan ekspresif.

```

<?php
declare(strict_types=1);

class PasienRumahSakit
{
    private string $rekamMedisId;
    private string $namaPasien;
    private int $umur;
    private float $suhuTubuh;
}

```

```

    public function __construct(string $id, string $nama, int $umur, float $suhu) {
        $this->setRekamMedisId($id);
        $this->setNamaPasien($nama);
        $this->setUmur($umur);
        $this->setSuhuTubuh($suhu);
    }

    public function getRekamMedisId(): string { return $this->rekamMedisId; }
    private function setRekamMedisId(string $id): void {
        if (!preg_match('/^RM-\d{5}$/', $id)) {
            throw new \InvalidArgumentException("Format ID Rekam Medis harus
RM-XXXXX!");
        }
        $this->rekamMedisId = $id;
    }

    public function getNamaPasien(): string { return $this->namaPasien; }
    public function setNamaPasien(string $nama): self {
        $bersih = trim($nama);
        if (strlen($bersih) < 3) {
            throw new \InvalidArgumentException("Nama pasien minimal 3 karakter!");
        }
        $this->namaPasien = $bersih;
        return $this; // FLuent Method Chaining
    }

    public function getUmur(): int { return $this->umur; }
    public function setUmur(int $umur): self {
        if ($umur < 0 || $umur > 130) {
            throw new \InvalidArgumentException("Rentang umur tidak realistis: {$umur}
tahun!");
        }
        $this->umur = $umur;
        return $this;
    }

    public function getSuhuTubuh(): float { return $this->suhuTubuh; }
    public function setSuhuTubuh(float $suhu): self {
        if ($suhu < 30.0 || $suhu > 45.0) {
            throw new \InvalidArgumentException("Pengukuran suhu di luar ambang medis
manusia!");
        }
        $this->suhuTubuh = $suhu;
        return $this;
    }
}

// Penggunaan FLuent Setter Chaining:
$pasien = new PasienRumahSakit("RM-10245", "Teuku Iskandar", 28, 36.6);
$pasien->setNamaPasien("Teuku Iskandar Muda")
->setUmur(29)
->setSuhuTubuh(37.1);

```

## 4.4 Konsep Kekelatan Data (Immutability): readonly di PHP 8.1 & 8.2

Dalam arsitektur perangkat lunak modern (seperti *Domain-Driven Design* / DDD), konsep *Immutability* (ketetapan nilai objek) memegang peranan krusial. Objek yang *immutable* (disebut *Value Object*) tidak dapat diubah status datanya setelah pertama kali

Hlm. 32



diinisialisasi di *constructor*. Sifat kekal ini mencegah *side-effects* tak terduga, aman saat diproses dalam konkurensi, dan memudahkan *debugging*.

PHP 8.1 memperkenalkan `readonly`, dan PHP 8.2 melengkapinya dengan `readonly class` yang secara otomatis menjadikan seluruh properti class bersifat `readonly` serta melarang pembuatan properti dinamis.

```
<?php
declare(strict_types=1);

// PHP 8.2: Readonly Class (Value Object Uang)
readonly class NilaiMataUang
{
    public function __construct(
        public float $nominal,
        public string $kodeMataUang = "IDR"
    ) {
        if ($nominal < 0) {
            throw new \InvalidArgumentException("Nominal uang tidak boleh negatif!");
        }
    }

    // Operasi matematika menghasilkan Objek BARU (tidak memodifikasi status objek Lama)
    public function tambah(NilaiMataUang $lain): self {
        if ($this->kodeMataUang !== $lain->kodeMataUang) {
            throw new \InvalidArgumentException("Mata uang harus sejenis!");
        }
        return new self($this->nominal + $lain->nominal, $this->kodeMataUang);
    }
}

$uang1 = new NilaiMataUang(50_000, "IDR");
$uang2 = new NilaiMataUang(25_000, "IDR");
$total = $uang1->tambah($uang2); // Objek baru dengan nominal Rp 75.000
```

## 4.5 Paradigma Mutakhir PHP 8.4: Asymmetric Visibility & Property Hooks

Rilis PHP 8.4 menghadirkan terobosan terbesar dalam evolusi enkapsulasi dengan dua fitur revolusioner:

### 4.5.1. Asymmetric Visibility (public private(set))

Seringkali kita menginginkan suatu properti dapat dibaca secara bebas oleh pihak luar (public read), namun perubahan nilainya hanya boleh dilakukan oleh internal class itu sendiri (private write). Di masa lalu, kita terpaksa mendeklarasikan properti `private` dan menuliskan getter method secara berulang-ulang.

Di PHP 8.4, hal ini diselesaikan secara native dengan sintaks `public private(set)` atau `public protected(set)`:

```
<?php
declare(strict_types=1);
```

```
// PHP 8.4: Asymmetric Visibility
class AnggotaPerpustakaan
{
    // Publik bebas membaca ($a->id, $a->totalPoin), tapi penulisan dikunci di
    // internal class:
    public private(set) string $idAnggota;
    public private(set) string $nama;
    public private(set) int $totalPoinAktivitas = 0;

    public function __construct(string $id, string $nama) {
        $this->idAnggota = $id;
        $this->nama      = $nama;
    }

    public function tambahPoinKunjungan(): void {
        $this->totalPoinAktivitas += 10; // ✅ Diizinkan: mutasi dari dalam class
    }
}

$mhs = new AnggotaPerpustakaan("MHS-001", "Cut Nyak Dhien");
echo $mhs->nama; // ✅ Boleh dibaca langsung! Output: Cut Nyak Dhien
echo $mhs->totalPoinAktivitas; // ✅ Boleh dibaca langsung! Output: 0
// $mhs->totalPoinAktivitas = 999; // ❌ FATAL ERROR: Cannot modify private(set)
// property from outside!
```

#### 4.5.2. Property Hooks (get dan set hooks)

PHP 8.4 memperkenalkan Property Hooks yang memungkinkan penyematan logika validasi, transformasi string, maupun komputasi virtual langsung pada definisi properti tanpa memerlukan method pembantu:

```
<?php
declare(strict_types=1);

// PHP 8.4: Property Hooks
class ProfilAkun
{
    // Backed Property dengan Hook Validasi Format Email
    public string $email {
        get => $this->email;
        set(string $nilaiBaru) {
            if (!filter_var($nilaiBaru, FILTER_VALIDATE_EMAIL)) {
                throw new \InvalidArgumentException("Format email tidak valid!");
            }
            $this->email = strtolower(trim($nilaiBaru));
        }
    }

    public string $namaDepan;
    public string $namaBelakang;

    // Virtual Property: Tidak memakan alokasi RAM, dikomputasi saat diakses
    public string $namaLengkap {
        get => "{$this->namaDepan} {$this->namaBelakang}";
    }
}
```

## 4.6 Magic Methods untuk Kendali Akses Properti Dinamis

PHP menyediakan magic methods khusus untuk mencegah (intercept) upaya pembacaan, penulisan, pemeriksaan, maupun penghapusan properti yang tidak dapat diakses langsung:

- **\_\_get(string \$name): mixed:** Mencegat pembacaan properti yang private atau belum didefinisikan.
- **\_\_set(string \$name, mixed \$value): void:** Mencegat penulisan nilai ke properti yang private atau belum terdefinisi.
- **\_\_isset(string \$name): bool:** Mencegat pemanggilan fungsi `isset()` atau `empty()` pada properti terenkapsulasi.
- **\_\_unset(string \$name): void:** Mencegat pemanggilan instruksi `unset()` pada properti terenkapsulasi.

### ⚠ Peringatan PHP 8.2+: Depreksi Dynamic Properties

Mulai PHP 8.2, penulisan properti dinamis tanpa deklarasi (`$obj->propertiBaru = 100;`) telah resmi di-deprecate dan memicu peringatan runtime. Seluruh properti wajib dideklarasikan secara eksplisit di dalam class, atau class tersebut harus ditandai dengan atribut `#[\AllowDynamicProperties]`.

### 🔒 Tips Keamanan Enkapsulasi: Terapkan Prinsip Least Privilege

Terapkan prinsip Least Privilege secara konsisten pada setiap anggota class. Awali seluruh properti data sebagai 'private'. Hanya naikan hak akses ke 'protected' jika memang disiapkan untuk diwariskan ke subclass, dan manfaatkan fitur 'public private(set)' pada PHP 8.4+ untuk mengekspos data publik yang terlindungi dari manipulasi luar.

## Rangkuman Materi

1. Enkapsulasi menyatukan data dan metode seraya melindungi status internal objek (Information Hiding).
2. State Invariant menjamin bahwa status data bisnis objek selalu valid dari lahir hingga selesai dieksekusi.
3. Prinsip 'Tell, Don't Ask' menolak Anemic Domain Model dengan menempatkan logika bisnis langsung pada class pemilik data.
4. Tiga level hak akses: public (terbuka), protected (keluarga warisan), dan private (terkunci rapat).
5. Fitur readonly (PHP 8.1/8.2) mewujudkan Value Object yang immutable dan aman dari efek samping konkurensi.
6. PHP 8.4 menghadirkan Asymmetric Visibility (`public private(set)`) dan Property Hooks (`get/set`) yang menyederhanakan penulisan kode enkapsulasi kelas industri.

### Soal Latihan & Penugasan Mandiri

1. Jelaskan mengapa pendekatan 'Tell, Don't Ask' lebih unggul dibandingkan membuat Getter dan Setter untuk setiap variabel tanpa aturan validasi!
2. Uraikan perbedaan fungsional antara `readonly property` di PHP 8.1 dengan `public private(set)` di PHP 8.4!
3. Rancanglah class `NilaiMahasiswa` dengan enkapsulasi ketat untuk properti `$tugas`, `$uts`, `$uas` (rentang 0-100), method `hitungNilaiAkhir()`, dan method `getHurufMutu()`!
4. Bagaimana cara mengamankan array mutasi transaksi perbankan agar tidak dapat dimodifikasi oleh kode pemanggil saat dikembalikan melalui getter (Defensive Copying)?

## BAB 5

## PEWARISAN (INHERITANCE), FINAL KEYWORD, DAN KOMPOSISI TRAIT

### Tujuan & Capaian Pembelajaran (Sub-CPMK 3)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami fondasi teoritis Inheritance (Pewarisan Sifat), prinsip Taksonomi 'Is-A', serta perbedaan antara Subtyping dan Code Reuse.
- Mengimplementasikan pewarisan menggunakan kata kunci `extends` dan mengelola siklus hidup inisialisasi menggunakan `parent::__construct()`.
- Menerapkan teknik Method Overriding untuk menyesuaikan perilaku subclass seraya mempertahankan integritas antarmuka induk.
- Mengendalikan dan mengamankan rancangan hirarki menggunakan kata kunci `final` pada class, method, dan class constants (PHP 8.1+).
- Mengatasi keterbatasan Single Inheritance di PHP menggunakan Trait (Horizontal Code Reuse), mengelola resolusi konflik (`instead of` dan `as`), serta memanfaatkan konstanta di dalam Trait (PHP 8.2+).
- Menganalisis kelemahan desain hierarki (Fragile Base Class Problem) dan menerapkan prinsip 'Favor Composition over Inheritance'.

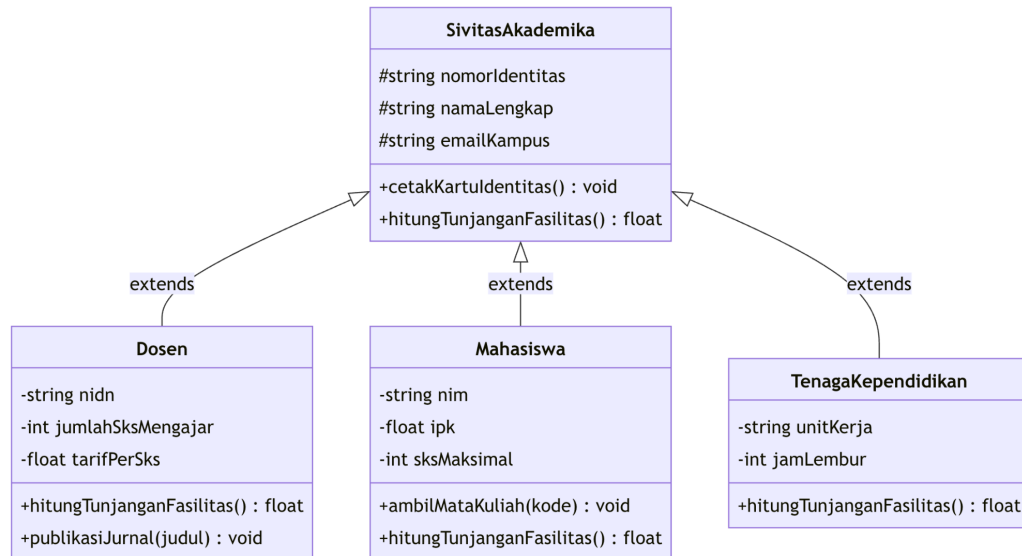
### 5.1 Filosofi dan Fondasi Teoritis Pewarisan

Dalam rekayasa perangkat lunak berorientasi objek, Pewarisan (*Inheritance*) merupakan pilar kedua yang memungkinkan suatu class baru (Subclass / Child Class) mengadopsi seluruh atribut (*state*) dan method (*behavior*) yang telah didefinisikan pada class yang sudah ada sebelumnya (Super Class / Parent Class).

Relasi pewarisan mencerminkan hubungan taksonomi 'Is-A' (Adalah Seorang / Adalah Sebuah): Dosen adalah seorang Sivitas Akademika, Mahasiswa adalah seorang Sivitas Akademika, dan Mobil Listrik adalah sebuah Kendaraan Bermotor. Melalui pewarisan, atribut dan perilaku umum yang berlaku untuk seluruh anggota taksonomi (seperti nomor identitas, nama lengkap, dan email) cukup dituliskan satu kali pada superclass. Hal ini secara langsung mewujudkan prinsip DRY (Don't Repeat Yourself) dan mempermudah pemeliharaan sistem. Untuk memperjelas konsep pewarisan dapat dilihat pada Gambar 4 yang memperjelas bagaimana pewarisan pada pembuatan Super Class (*parent class*) dan subclass(*child class*).

Namun, seorang arsitek perangkat lunak wajib mewaspadaai fenomena Fragile Base Class Problem. Ketika pohon pewarisan dibuat terlalu dalam (misalnya lebih dari 3 atau 4 tingkat hierarki), sistem menjadi sangat rapuh: modifikasi kecil pada kode internal superclass dapat memicu kerusakan berantai (*ripple effects*) pada puluhan subclass di

bawahnya. Oleh karena itu, standar industri merekomendasikan prinsip '*Favor Composition over Inheritance*' (Utamakan Komposisi dan Trait dibandingkan Pewarisan Kelas yang Terlalu Dalam).



Gambar 4. Fondasi dan Filosofis Pewarisan (Inheritance)

## 5.2 Anatomi Pewarisan di PHP: `extends`, `parent::`, dan Method Overriding

Di dalam PHP, proses penurunan sifat dinyatakan dengan kata kunci `extends`. Ketika sebuah subclass mendefinisikan constructor-nya sendiri, *Zend Engine* tidak secara otomatis memanggil constructor milik superclass. Oleh sebab itu, pengembang wajib memanggil constructor induk secara eksplisit menggunakan sintaks `parent::__construct(...)`.

*Method Overriding* adalah kemampuan subclass untuk menulis ulang implementasi method yang diwarisinya dari parent class agar sesuai dengan kebutuhan spesifik subclass bersangkutan, seraya tetap dapat memanggil fungsionalitas dasar parent melalui `parent::namaMethod()`.

```

<?php
declare(strict_types=1);

namespace App\Domain;

// Superclass (Parent Class)
class SivitasAkademika
{
    public function __construct(
        protected readonly string $nomorIdentitas,
        protected string $namaLengkap,
        protected string $emailKampus
    ) {
  
```

```

        if (empty($nomorIdentitas) || empty($namaLengkap)) {
            throw new \InvalidArgumentException("Nomor Identitas dan Nama wajib
diisi!");
        }
    }

    public function cetakKartuIdentitas(): void {
        echo "=====\n";
        echo "KARTU SIVITAS AKADEMIKA UUI\n";
        echo "Nomor ID : {$this->nomorIdentitas}\n";
        echo "Nama      : {$this->namaLengkap}\n";
        echo "Email     : {$this->emailKampus}\n";
        echo "Peran      : " . static::class . "\n";
    }

    public function hitungBantuanFasilitas(): float {
        return 100_000.0; // Bantuan kuota internet dasar universitas
    }
}

// Subclass Dosen (Mewarisi SivitasAkademika)
class Dosen extends SivitasAkademika
{
    public function __construct(
        string $nidn,
        string $namaLengkap,
        string $emailKampus,
        private string $jabatanFungsional = "Asisten Ahli",
        private int $sksMengajar = 12
    ) {
        // 1. Constructor Chaining: Inisialisasi properti parent
        parent::__construct($nidn, $namaLengkap, $emailKampus);
    }

    // 2. Method Overriding: Menyesuaikan kalkulasi tunjangan khusus dosen
    public function hitungBantuanFasilitas(): float {
        $dasar = parent::hitungBantuanFasilitas(); // Mengambil nilai Rp 100.000 dari
parent
        $tunjanganSks = $this->sksMengajar * 50_000.0;
        return $dasar + $tunjanganSks;
    }

    public function cetakKartuIdentitas(): void {
        parent::cetakKartuIdentitas();
        echo "Jabatan Fungsional : {$this->jabatanFungsional}\n";
        echo "Beban Mengajar      : {$this->sksMengajar} SKS\n";
        echo "Total Bantuan Fas. : Rp " .
number_format($this->hitungBantuanFasilitas(), 0, ',', '.') . "\n";
        echo "=====\n";
    }
}

```

### 5.3 Pengendalian Hierarki dengan Kata Kunci `final`

Kata kunci final digunakan untuk mengunci elemen arsitektur agar tidak dapat diperluas atau diubah oleh pengembang lain. PHP mendukung tiga tingkat penguncian final:

1. **final class:** Melarang class dijadikan superclass bagi class lain mana pun. Sangat berguna untuk class konfigurasi keamanan atau utilitas tertutup.

2. **final method:** Melarang method tertentu di-override oleh subclass turunan. Pola ini merupakan inti dari Template Method Pattern.
3. **final class constant (PHP 8.1+):** Mencegah nilai konstanta class ditimpa (overridden) oleh subclass turunan.

```
<?php
declare(strict_types=1);

// Contoh final class: Terkunci rapat dari pewarisan
final class EnkripsiKeamananToken
{
    // PHP 8.1: final constant
    final public const ALGORITMA = "AES-256-GCM";
}

class AturanKelulusanProdi
{
    // final method: Rumus penentuan yudisium dilarang diutak-atik subclass
    final public function validasiYudisium(float $ipk, int $totalSks): bool {
        return $ipk >= 2.00 && $totalSks >= 144;
    }
}
```

## 5.4 Trait: Solusi Horizontal Code Reuse di PHP

PHP menganut model *Single Inheritance* (setiap class hanya boleh memiliki satu parent langsung). Keterbatasan ini sering menimbulkan kesulitan ketika kita ingin berbagi fungsionalitas umum (misalnya *logging*, serialisasi JSON, *soft deletion*) ke banyak class yang tidak berada dalam satu pohon taksonomi warisan.

Trait (diteliti oleh Scharli et al., 2003) menyediakan mekanisme *Horizontal Code Reuse* di mana potongan-potongan perilaku dapat disisipkan ke berbagai class independen menggunakan instruksi use.

```
<?php
declare(strict_types=1);

namespace App\Traits;

trait AuditLoggableTrait
{
    public function catatAudit(string $aktivitas): void {
        $waktu = date('Y-m-d H:i:s');
        echo "[AUDIT LOG] [{$waktu}] [ " . static::class . " ] {$aktivitas}\n";
    }
}

trait ExportableJsonTrait
{
    public function keJson(): string {
        return json_encode(get_object_vars($this), JSON_PRETTY_PRINT |
JSON_UNESCAPED_UNICODE);
    }
}

// PHP 8.2+: Konstanta di dalam Trait
trait CacheConfigTrait
```



```
{
    public const DEFAULT_TTL = 3600; // 1 Jam
}
```

## 5.5 Resolusi Konflik Trait: `insteadof` dan `as`

Ketika sebuah class mengimpor dua Trait berbeda yang kebetulan memiliki nama method yang sama, Zend Engine akan memicu Fatal Error akibat bentrok nama (Name Collision). PHP menyediakan operator `insteadof` (untuk memilih implementasi mana yang menang) dan operator `as` (untuk memberikan nama alias pada implementasi yang kalah):

```
<?php
declare(strict_types=1);

trait EmailNotifierTrait {
    public function kirimPesan(string $teks): void {
        echo "✉ Mengirim Email: {$teks}\n";
    }
}

trait TelegramNotifierTrait {
    public function kirimPesan(string $teks): void {
        echo "📠 Mengirim Bot Telegram: {$teks}\n";
    }
}

class PusatNotifikasi
{
    use EmailNotifierTrait, TelegramNotifierTrait {
        // 1. Pilih TelegramNotifierTrait untuk method kirimPesan utama:
        TelegramNotifierTrait::kirimPesan insteadof EmailNotifierTrait;

        // 2. Beri nama alias untuk EmailNotifierTrait agar tetap dapat dipanggil:
        EmailNotifierTrait::kirimPesan as kirimEmail;
    }
}

$notif = new PusatNotifikasi();
$notif->kirimPesan("Server pulih kembali."); // Memanggil Telegram Notifier
$notif->kirimEmail("Laporan rekapitulasi."); // Memanggil Email Notifier
```

## 5.6 Abstract Method di dalam Trait

Trait juga dapat mendeklarasikan Abstract Method untuk menuntut class pengguna menyediakan method atau data tertentu agar logika internal Trait dapat berjalan dengan sempurna:

```
<?php
trait NomorSuratOtomatisTrait
{
    // Menuntut class pengguna mengembalikan kode unit
    abstract public function getKodeUnit(): string;

    public function buatNomorSurat(int $urutan): string {
        $tahun = date('Y');
```

```

        return sprintf("%04d/UUI-%s/%s", $urutan, $this->getKodeUnit(), $tahun);
    }
}

class SuratKeputusanDekan
{
    use NomorSuratOtomatisTrait;

    public function getKodeUnit(): string {
        return "FST-DEKAN";
    }
}

```



#### Tips Desain Pewarisan: Pedoman Komposisi vs Pewarisan

Gunakan Pewarisan Kelas (extends) hanya jika relasi 'Is-A' terpenuhi secara murni dan tidak ada pelanggaran Liskov Substitution Principle. Untuk fungsionalitas utilitas lintas divisi yang bersifat modular (seperti logging, caching, hashing), selalu utamakan penggunaan Trait atau Dependency Injection.

### Rangkuman Materi

1. Pewarisan ('extends') membangun relasi taksonomi 'Is-A' dan memaksimalkan penggunaan ulang kode (DRY).
2. Sintaks `parent::__construct()` wajib dipanggil saat subclass mendefinisikan constructor-nya sendiri.
3. Method Overriding memungkinkan penyesuaian perilaku induk seraya memperluas fungsionalitas.
4. Kata kunci `final` mengunci class, method, dan konstanta (PHP 8.1+) dari risiko modifikasi liar.
5. Trait menyediakan solusi Horizontal Code Reuse untuk mengatasi batasan Single Inheritance di PHP.
6. Operator 'insteadof' dan 'as' menyelesaikan konflik bentrok nama method antar-trait secara presisi.

### Soal Latihan & Penugasan Mandiri

1. Jelaskan mengapa hirarki pewarisan yang terlalu dalam memicu Fragile Base Class Problem!
2. Bagaimana cara menyelesaikan bentrok nama method ketika sebuah class menggunakan dua Trait yang memiliki nama fungsi identik?
3. Rancanglah hirarki class: Parent `Kendaraan`, Child `MobilListrik` (menambahkan baterai dan kalkulasi jarak tempuh) yang dilengkapi Trait `AuditLoggableTrait`!
4. Kapan sebuah konstanta class sebaiknya ditandai dengan kata kunci `final const` di PHP 8.1+?

## BAB 6

# POLIMORFISME (POLYMORPHISM) DAN DYNAMIC DISPATCH

---

### Tujuan & Capaian Pembelajaran (Sub-CPMK 3)

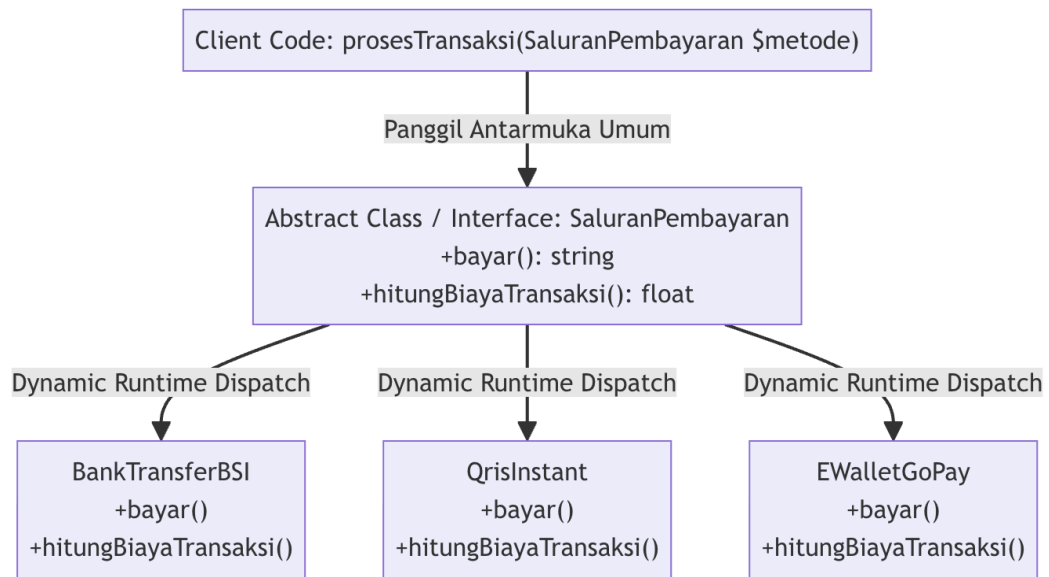
Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami filosofi fundamental Polymorphism (Polimorfisme), taksonomi teori tipe Cardelli & Wegner, serta prinsip 'Satu Antarmuka, Banyak Perilaku'.
- Memahami mekanisme eksekusi Dynamic Method Dispatch pada Zend Engine runtime PHP.
- Mengimplementasikan Polymorphic Type Hinting dan memproses Koleksi Objek Polimorfik (Polymorphic Collections).
- Membedakan secara analitis Polimorfisme berbasis Pewarisan Class (Class Inheritance) vs Polimorfisme berbasis Kontrak (Interface-based Polymorphism).
- Menggunakan operator instanceof secara tepat untuk Type Narrowing seraya menghindari Anti-pattern Type Checking.
- Merancang arsitektur perangkat lunak yang mematuhi Open/Closed Principle (OCP) dan Liskov Substitution Principle (LSP).

## 6.1 Filosofi dan Fondasi Teoritis Polimorfisme

Secara etimologi bahasa Yunani, Polimorfisme berasal dari kata *poly* (banyak) dan *morph* (bentuk atau rupa). Dalam ilmu rekayasa perangkat lunak berorientasi objek, polimorfisme adalah prinsip kemampuan objek-objek dari berbagai class turunan yang berbeda untuk merespons pemanggilan pesan atau method yang sama dengan cara dan implementasi unik mereka masing-masing.

Kekuatan utama polimorfisme terletak pada pemisahan yang tegas antara 'Apa yang harus dilakukan' (didefinisikan pada antarmuka umum / superclass) dan 'Bagaimana cara melakukannya' (didefinisikan secara spesifik oleh masing-masing subclass). Kode pemanggil (Client Code) cukup berkomunikasi dengan tipe acuan abstrak tanpa perlu mengetahui secara detail class konkrit mana yang sedang aktif saat runtime.



Gambar 5. Fondasi Teoritis Polimorfisme

Dalam literatur klasik teori tipe komputasi (Cardelli & Wegner, 1985), polimorfisme pada OOP bertumpu pada Subtyping / Inclusion Polymorphism, di mana sebuah variabel ber-tipe superclass dapat menampung objek dari subclass mana pun yang sah dan secara otomatis mengeksekusi perilaku yang tepat tanpa memerlukan percabangan logika if-else.

## 6.2 Mekanisme Dynamic Method Dispatch pada PHP Runtime

Di dalam PHP, resolusi pemanggilan method (`$objek->bayar()`) diselesaikan secara dinamis pada saat program berjalan (runtime). Ketika method dipanggil, Zend Engine memeriksa Virtual Method Table (V-Table) atau tabel simbol kelas dari instance objek yang tersimpan di memori dan melompat langsung ke alamat fungsi yang sesuai.

```

<?php
declare(strict_types=1);

namespace App\Pembayaran;

// Superclass Abstraksi
abstract class SaluranPembayaran
{
    public function __construct(protected float $totalTagihan) {}

    abstract public function bayar(): string;
    abstract public function hitungBiayaAdmin(): float;
}

// Subclass 1: Transfer Bank
class TransferBank extends SaluranPembayaran
{
    public function __construct(float $total, private string $namaBank, private string $rekening) {

```

```

        parent::__construct($total);
    }

    public function hitungBiayaAdmin(): float { return 4_000.0; }

    public function bayar(): string {
        $totalBayar = $this->totalTagihan + $this->hitungBiayaAdmin();
        return "🏦 [TRANSFER BANK] {$this->namaBank} ({$this->rekening}) | Total: Rp " .
            number_format($totalBayar, 0, ',', '.') . " (Termasuk Biaya Admin: Rp 4.000)";
    }
}

// Subclass 2: QRIS Instant
class QrisInstant extends SaluranPembayaran
{
    public function __construct(float $total, private string $merchantNMID) {
        parent::__construct($total);
    }

    public function hitungBiayaAdmin(): float { return $this->totalTagihan * 0.007; }
    // 0.7%

    public function bayar(): string {
        $totalBayar = $this->totalTagihan + $this->hitungBiayaAdmin();
        return "📱 [QRIS INSTANT] NMID: {$this->merchantNMID} | Total: Rp " .
            number_format($totalBayar, 0, ',', '.') . " [LUNAS REALTIME]";
    }
}

// Subclass 3: Dompot Digital
class EWalletGoPay extends SaluranPembayaran
{
    public function __construct(float $total, private string $nomorHp) {
        parent::__construct($total);
    }

    public function hitungBiayaAdmin(): float { return 1_000.0; }

    public function bayar(): string {
        $totalBayar = $this->totalTagihan + $this->hitungBiayaAdmin();
        return "💳 [E-WALLET GOPAY] Akun {$this->nomorHp} terdebet Rp " .
            number_format($totalBayar, 0, ',', '.') . " [SUKSES]";
    }
}

```

### 6.3 Polymorphic Type Hinting & Koleksi Polimorfik

Dengan Polymorphic Type Hinting, fungsi proses transaksi hanya perlu bergantung pada superclass SaluranPembayaran. Fungsi ini secara otomatis mematuhi Open/Closed Principle (terbuka untuk penambahan saluran baru, namun tertutup dari modifikasi kode lama).

```

<?php
// Fungsi Konsumen Polimorfik
function prosesTransaksiKasir(SaluranPembayaran $saluran): void {
    echo "Menghubungkan ke payment gateway...\n";
}

```

```

    $struk = $saluran->bayar(); // Dynamic Dispatch mengeksekusi implementasi child
    yang tepat
    echo $struk . "\n";
    echo "Biaya Admin: Rp " . number_format($saluran->hitungBiayaAdmin(), 0, ',', '.')
    . "\n";
    echo "-----\n";
}

// Array Polimorfik: Berisi ragam subclass berbeda dalam satu wadah seragam
$antreanPembayaran = [
    new TransferBank(500_000.0, "Bank Syariah Indonesia (BSI)", "7123456789"),
    new QrisInstant(25_000.0, "ID1020304050"),
    new EWalletGoPay(75_000.0, "081269001122"),
    new QrisInstant(150_000.0, "ID1020304050")
];

foreach ($antreanPembayaran as $transaksi) {
    prosesTransaksiKasir($transaksi);
}

```

## 6.4 Matriks Analisis: Polimorfisme Class vs Interface

| Dimensi Analisis  | Polimorfisme Berbasis Class<br>(extends)    | Polimorfisme Berbasis Interface<br>(implements)  |
|-------------------|---------------------------------------------|--------------------------------------------------|
| Relasi Konseptual | Hubungan keluarga taksonomi ketat (Is-A)    | Hubungan kontrak kemampuan perilaku (Can-Do)     |
| Pewarisan State   | Mewarisi properti dan method konkrit parent | Murni kontrak antarmuka tanpa state bersama      |
| Fleksibilitas     | Terikat batas Single Inheritance (1 parent) | Bebas diimplementasikan banyak class (Multiple)  |
| Standar Industri  | Hierarki entitas domain inti                | Desain Clean Architecture & Dependency Injection |

## 6.5 Type Narrowing Menggunakan Operator `instanceof`

Operator `instanceof` digunakan ketika kode perlu memeriksa tipe objek spesifik sebelum menjalankan operasi yang hanya ada pada subclass tertentu. Namun, perancang sistem harus menghindari penggunaan `instanceof` yang berlebihan (Anti-pattern Type Checking), karena hal tersebut mengindikasikan bahwa logika seharusnya ditempatkan ke dalam method polimorfik milik subclass.

```

<?php
class AuditorKeuangan
{
    public function audit(SaluranPembayaran $saluran): void {

```

```

        echo "Audit Transaksi: " . $saluran->bayar() . "\n";
    if ($saluran instanceof QrisInstant) {
        echo "❗ Validasi signature QRIS dengan Bank Indonesia.\n";
    } elseif ($saluran instanceof TransferBank) {
        echo "❗ Rekonsiliasi mutasi rekening koran perbankan.\n";
    }
    }
}

```

### 🎯 Tips Polimorfisme: Desain Berbasis Kontrak untuk Kemudahan Pengujian

Gunakan Polymorphic Type Hinting pada seluruh service layer aplikasi Anda. Dengan mengandalkan interface atau tipe abstrak, Anda dapat dengan mudah membuat Mock Object (objek tiruan) saat menjalankan Automated Unit Testing (PHPUnit) tanpa perlu menghubungkan sistem ke database atau payment gateway nyata.

## Rangkuman Materi

1. Polimorfisme ('Satu Antarmuka, Banyak Wujud') memisahkan antarmuka umum dari implementasi spesifik.
2. Dynamic Method Dispatch pada Zend Engine menyelesaikan pemanggilan method secara dinamis di runtime.
3. Polymorphic Type Hinting memungkinkan penulisan kode modular yang mematuhi Open/Closed Principle.
4. Polimorfisme berbasis Interface memberikan fleksibilitas tertinggi tanpa batasan Single Inheritance.
5. Operator instanceof menjamin Type Safety ketika diperlukan inspeksi tipe objek spesifik.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan bagaimana polimorfisme memfasilitasi penambahan metode pembayaran Cryptocurrency baru tanpa memodifikasi kode kasir yang sudah ada!
2. Apa perbedaan mendasar antara Polimorfisme Berbasis Pewarisan Class vs Polimorfisme Berbasis Interface?
3. Rancanglah sebuah sistem notifikasi polimorfik (Email, SMS, WhatsApp) dengan antarmuka `kirim(string \$tujuan, string \$pesan)`!
4. Mengapa terlalu banyak menggunakan percabangan `if (\$obj instanceof X)` di dalam alur logika bisnis dianggap sebagai code smell?

## BAB 7

# ABSTRAKSI: ABSTRACT CLASS, INTERFACE, DAN BACKED ENUM

### Tujuan & Capaian Pembelajaran (Sub-CPMK 3)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami filosofi fundamental Abstraction (Abstraksi) sebagai pilar ke-4 OOP, konsep Separation of Interface and Implementation, serta Design by Contract (DbC).
- Mendeklarasikan dan menerapkan Abstract Class serta Abstract Method menggunakan kata kunci `abstract` dan merancang pola Template Method Pattern.
- Merancang kontrak antarmuka murni menggunakan Interface, kata kunci `implements`, serta pewarisan antar-interface (Interface Inheritance).
- Mengimplementasikan Multiple Interfaces pada sebuah Class untuk mengatasi keterbatasan pewarisan tunggal.
- Membedakan secara tajam kapan harus menggunakan Abstract Class (IS-A) vs Interface (CAN-DO) dalam arsitektur perangkat lunak skala enterprise.
- Mengintegrasikan fitur modern Backed Enum (PHP 8.1+) dengan method fungsional dan pattern matching (`match`) untuk manajemen status yang type-safe.

## 7.1 Filosofi dan Fondasi Teoritis Abstraksi

Dalam rekayasa perangkat lunak, Abstraksi (Abstraction) adalah proses menyederhanakan kompleksitas sistem dengan hanya menampilkan karakteristik dan antarmuka penting kepada dunia luar, seraya menyembunyikan mekanisme teknis internal yang rumit.

Sebagai analogi dunia nyata, ketika seseorang mengemudikan mobil, ia cukup berinteraksi dengan pedal gas, pedal rem, dan roda kemudi. Pengemudi tidak perlu mengetahui secara mikroskopis rasio kompresi bahan bakar di dalam ruang silinder atau perpindahan fluida transmisi hidrolik. Antarmuka pedal menyederhanakan kompleksitas mesin tersebut.

Dalam bahasa PHP modern, pilar abstraksi diwujudkan melalui dua konstruksi utama: Abstract Class (kerangka dasar setengah jadi untuk hierarki keluarga erat IS-A) dan Interface (kontrak perilaku murni untuk kemampuan lintas modul CAN-DO).

## 7.2 Abstract Class & Template Method Pattern

Abstract Class adalah class induk yang tidak dapat diinstansiasi langsung menggunakan operator `new`. Class ini memuat gabungan antara Concrete Method



(method yang sudah memiliki kode fungsional teruji) dan Abstract Method (method tanpa badan fungsi yang wajib disempurnakan oleh subclass turunan).

Pola desain Template Method Pattern mengunci alur kerja utama (master workflow) pada parent class menggunakan kata kunci final, sementara langkah-langkah detailnya diserahkan kepada subclass melalui abstract method:

```
<?php
declare(strict_types=1);

namespace App\Laporan;

// Abstract Superclass
abstract class TemplateLaporanAkademik
{
    public function __construct(
        protected string $judulLaporan,
        protected string $semester
    ) {}

    // 1. Concrete Method: Kop Surat Standar Universitas
    public function cetakKopSurat(): void {
        echo "=====\n";
        echo "UNIVERSITAS UBUDIYAH INDONESIA\n";
        echo "FAKULTAS SAINS DAN TEKNOLOGI - PROGRAM STUDI INFORMATIKA\n";
        echo "Judul Laporan : {$this->judulLaporan}\n";
        echo "Semester      : {$this->semester}\n";
        echo "-----\n";
    }

    // 2. Abstract Methods: Wajib disediakan oleh subclass
    abstract protected function ambilSumberData(): array;
    abstract protected function susunBadanLaporan(array $data): string;
    abstract public function exportFormat(): string;

    // 3. Template Method (Final): Alur kerja utama terkunci aman
    final public function generateDokumen(): void {
        $this->cetakKopSurat();
        $data = $this->ambilSumberData();
        $konten = $this->susunBadanLaporan($data);
        echo $konten . "\n";
        echo "Format Dokumen: " . $this->exportFormat() . "\n";
        echo "=====\n";
    }
}

// Subclass Konkrit: Rekapitulasi IPK Mahasiswa
class LaporanIpMahasiswa extends TemplateLaporanAkademik
{
    protected function ambilSumberData(): array {
        return [
            ['nim' => '240101', 'nama' => 'Cut Meurah Intan', 'ipk' => 3.92],
            ['nim' => '240102', 'nama' => 'Teuku Rayhan', 'ipk' => 3.85]
        ];
    }

    protected function susunBadanLaporan(array $data): string {
        $out = "REKAPITULASI IPK MAHASISWA:\n";
        foreach ($data as $mhs) {
            $out .= sprintf("• [%s] %-20s : IPK %.2f\n", $mhs['nim'], $mhs['nama'],
                $mhs['ipk']);
        }
    }
}
```

```

    }
    return $out;
}

public function exportFormat(): string {
    return "Dokumen Portabel (PDF A4 Landscape)";
}
}

```

### 7.3 Interface: Kontrak Perilaku Murni (\*CAN-DO\*)

Interface adalah kontrak murni tanpa properti data dan tanpa implementasi method. Seluruh method yang dideklarasikan di dalam interface otomatis bersifat public abstract. Sebuah class dapat mengimplementasikan banyak interface sekaligus (Multiple Interface Implementation).

```

<?php
declare(strict_types=1);

namespace App\Kontrak;

interface ExportablePdfInterface { public function renderPdf(): string; }
interface KirimEmailInterface { public function kirimEmail(string $tujuan): bool; }
interface AuditLoggableInterface { public function catatLog(string $pesan): void; }

// Implementasi Multiple Interfaces
class BerkasTranskripNilai implements ExportablePdfInterface, KirimEmailInterface,
AuditLoggableInterface
{
    public function __construct(public readonly string $nim, public readonly string
    $nama) {}

    public function renderPdf(): string {
        return "[PDF-BINARY] Transkrip Resmi {$this->nama} ({$this->nim}) siap
        diunduh.";
    }

    public function kirimEmail(string $tujuan): bool {
        echo "📧 Mengirim transkrip ke <{$tujuan}>... Berhasil!\n";
        $this->catatLog("Transkrip dikirim ke {$tujuan}");
        return true;
    }

    public function catatLog(string $pesan): void {
        echo "📝 [LOG] " . date('Y-m-d H:i:s') . " - {$pesan}\n";
    }
}

```

### 7.4 Matriks Komparasi: Abstract Class vs Interface

| Parameter Analisis | Abstract Class           | Interface              |
|--------------------|--------------------------|------------------------|
| Kata Kunci         | abstract class + extends | interface + implements |

| Relasi Konseptual   | IS-A (Identitas kekeluargaan erat)   | CAN-DO (Kontrak kemampuan/perilaku)     |
|---------------------|--------------------------------------|-----------------------------------------|
| Pewarisan Ganda     | ❌ Dilarang (Single Inheritance)      | ✅ Diizinkan (Multiple Implementation)   |
| Properti / State    | ✅ Boleh (public, protected, private) | ❌ Dilarang (Hanya konstanta const)      |
| Implementasi Method | Campuran (Bisa konkret + abstract)   | Murni deklarasi tanpa kurung kurawal {} |
| Constructor         | ✅ Bisa memiliki __construct()        | ❌ Tidak boleh memiliki constructor      |

## 7.5 Backed Enum di PHP 8.1+ dan Pattern Matching

PHP 8.1 memperkenalkan Backed Enum (tipe data enumerasi yang nilainya terikat pada string atau integer). Backed Enum sangat ideal dipadukan dengan Interface dan Abstract Class untuk mengelola status sistem secara type-safe serta mendukung ekspresi match:

```
<?php
declare(strict_types=1);

namespace App\Enums;

interface LabelledEnumInterface {
    public function getLabel(): string;
    public function getBadgeColor(): string;
}

enum StatusKelulusan: string implements LabelledEnumInterface
{
    case LULUS_CUMLAUDE = 'CUMLAUDE';
    case LULUS_MEMUASKAN = 'MEMUASKAN';
    case BERSYARAT = 'BERSYARAT';
    case MENGULANG = 'MENGULANG';

    public function getLabel(): string {
        return match($this) {
            self::LULUS_CUMLAUDE => 'Lulus dengan Pujian (Cum Laude)',
            self::LULUS_MEMUASKAN => 'Lulus Sangat Memuaskan',
            self::BERSYARAT => 'Lulus Bersyarat (Revisi Skripsi)',
            self::MENGULANG => 'Wajib Mengulang Sidang',
        };
    }

    public function getBadgeColor(): string {
        return match($this) {
            self::LULUS_CUMLAUDE => '#10B981',
            self::LULUS_MEMUASKAN => '#3B82F6',
            self::BERSYARAT => '#F59E0B',
            self::MENGULANG => '#EF4444',
        };
    }
}
```

```

    };
}
}

$status = StatusKelulusan::from('CUMLAUDE');
echo "Predikat : " . $status->getLabel() . "\n";
echo "Warna Tag: " . $status->getBadgeColor() . "\n";

```

#### 🎯 Tips Abstraksi: Prinsip 'Program to an Interface, not an Implementation'

Rancanglah lapisan bisnis aplikasi Anda dengan selalu bergantung pada Interface (abstraksi), bukan pada class konkrit. Dengan cara ini, Anda dapat mengganti pustaka pihak ketiga (misal: mengganti Mailgun dengan AWS SES, atau mengganti MySQL dengan PostgreSQL) tanpa perlu mengubah kode domain aplikasi sama sekali.

## Rangkuman Materi

1. Abstraksi menyembunyikan detail teknis yang rumit dan menyajikan antarmuka esensial kepada pengguna.
2. Abstract Class memadukan concrete methods dan abstract methods dalam pola Template Method Pattern.
3. Interface adalah kontrak murni tanpa state data, memungkinkan Multiple Interfaces pada class.
4. Pewarisan antar-interface (Interface Inheritance) dapat dilakukan menggunakan kata kunci extends.
5. Backed Enum (PHP 8.1+) menghadirkan type safety untuk status data dengan dukungan method dan match expression.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan perbedaan mendasar antara relasi 'IS-A' pada Abstract Class dan 'CAN-DO' pada Interface!
2. Bagaimana cara kerja Template Method Pattern dalam menjaga integritas master workflow sebuah laporan akademik?
3. Rancanglah sebuah Backed Enum `StatusPembayaran` (PENDING, PAID, EXPIRED, FAILED) yang mengimplementasikan method `isFinal(): bool`!
4. Mengapa sebuah Interface dilarang memiliki properti variabel dan constructor?

## BAB 8

# MANAJEMEN NAMESPACE, STANDAR PSR-4, DAN COMPOSER AUTOLOADING

---

### Tujuan & Capaian Pembelajaran (Sub-CPMK 4)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami urgensi dan filosofi Namespace untuk mencegah polusi ruang lingkup global (Global Namespace Pollution) dan tabrakan nama class (Name Collision).
- Menguasai sintaks deklarasi namespace, aturan resolusi nama simbol (Fully Qualified, Qualified, Unqualified), serta penggunaan instruksi use, aliasing (as), dan Group Use.
- Memahami evolusi mekanisme Autoloading di PHP dari era `require_once`, `spl_autoload_register()`, hingga standar industri modern PSR-4 (PHP Standards Recommendation).
- Mengonfigurasi berkas `composer.json` untuk mengelola pemetaan namespace aplikasi (`autoload.psr-4`) dan lingkungan pengujian (`autoload-dev.psr-4`).
- Menguasai teknik optimasi performa `ClassLoader` pada server produksi (`composer dump-autoload -o` dan `--classmap-authoritative`).
- Menyusun struktur direktori proyek PHP skala besar berstandar arsitektur industri (Clean / Layered Architecture).

## 8.1 Filosofi Namespace: Mengatasi Polusi Lingkup Global

Pada masa awal perkembangan PHP (sebelum versi 5.3), seluruh class, fungsi, dan konstanta yang didefinisikan akan ditempatkan di dalam satu wadah global yang sama (Global Scope). Hal ini memicu masalah kritis ketika dua pustaka berbeda mendefinisikan nama class yang sama, menghasilkan Fatal Error: Cannot redeclare class.

Untuk mencegah tabrakan nama tersebut, para pengembang zaman dahulu terpaksa memberi awalan nama class yang sangat panjang (misalnya Zend Service Amazon Ec2 Instance Configuration). Namespace menyediakan mekanisme pembungkus virtual hierarkis yang mengisolasi simbol-simbol kode ke dalam ruang nama masing-masing, mirip dengan struktur direktori di dalam sistem operasi komputer.

## 8.2 Anatomi Deklarasi dan Resolusi Simbol Namespace

PHP mengenali tiga tingkatan cara penulisan nama class atau simbol:

1. **Fully Qualified Name (FQN):** Dimulai dengan garis miring terbalik (`\`), menunjuk langsung dari akar global (contoh: `\App\Domain\Model\Mahasiswa`).
2. **Qualified Name:** Mengandung garis miring terbalik di tengah namun tidak diawali `\` (contoh: `Model\Mahasiswa`).

- 3. Unqualified Name:** Hanya menuliskan nama class secara langsung tanpa garis miring terbalik (contoh: Mahasiswa).

```
<?php
declare(strict_types=1);

namespace App\Controller;

// 1. Mengimpor Class Tunggal dengan Alias untuk Mencegah Bentrok Nama
use App\Akademik\Mahasiswa as MahasiswaAkademik;
use App\Penerimaan\Mahasiswa as MahasiswaPendaftar;

// 2. Mengimpor Fungsi dan Konstanta Spesifik
use function App\Helper\formatMataUang;
use const App\Config\MAX_SKS_SEMESTER;

// 3. Group Use Statements (PHP 7+)
use App\Services\{
    RegistrasiKrsService,
    ValidasiSyaratYudisiumService,
    AuditLogService
};

// 4. Mengakses Global PHP Built-in Classes
use DateTimeImmutable;
use InvalidArgumentException;

class PendaftaranController
{
    public function prosesPendaftaran(): void {
        $mhsBaru = new MahasiswaPendaftar("REG-2025-001", "Cut Meurah");
        $mhsAktif = new MahasiswaAkademik("240101001", "Teuku Iskandar");

        $waktu = new DateTimeImmutable();
        echo "Pendaftaran diproses pada: " . $waktu->format('Y-m-d H:i:s') . "\n";
    }
}
```

### 8.3 Standar Autoloading: Dari `require` ke Standar PSR-4

Dahulu, setiap class harus disertakan secara manual menggunakan `require_once`, yang mengakibatkan fenomena Spaghetti Includes dan membebani performa I/O disk server. PHP kemudian memperkenalkan `spl_autoload_register()` yang memungkinkan pendaftaran fungsi penangkap (callback) otomatis ketika suatu class belum dimuat di memori.

Konsorsium PHP-FIG (PHP Standards Recommendation) menetapkan standar PSR-4 yang mengatur pemetaan matematis antara Namespace terstruktur dengan direktori fisik file di server: Namespace Prefix `App\` dipetakan ke direktori `src/`, sehingga class `App\Domain\Model\Mahasiswa` secara otomatis dimuat dari berkas fisik `src/Domain/Model/Mahasiswa.php`.

## 8.4 Konfigurasi Autoloading Menggunakan Composer

Composer adalah Dependency & Package Manager resmi PHP yang mengelola autoloader PSR-4 secara otomatis melalui berkas konfigurasi composer.json:

```
// composer.json
{
    "name": "uui/sistem-informasi-akademik",
    "description": "Sistem Informasi Akademik Berbasis OOP PHP 8+",
    "type": "project",
    "license": "MIT",
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        },
        "files": [
            "src/Helper/GlobalFunctions.php"
        ]
    },
    "autoload-dev": {
        "psr-4": {
            "Tests\\": "tests/"
        }
    }
}
```

### Perintah Kunci Manajemen Autoloader Composer:

1. `composer dump-autoload`: Memperbarui peta berkas autoload saat ada penambahan file atau perubahan namespace.
2. `composer dump-autoload -o (--optimize)`: Menghasilkan tabel hash ClassMap Level 1 untuk performa tinggi di lingkungan staging/produksi.
3. `composer dump-autoload -a (--classmap-authoritative)`: Mengunci ClassMap secara absolut dan melarang pencarian sistem berkas dinamis saat runtime, menghasilkan kecepatan akses kelas maksimal.

## 8.5 Struktur Standar Proyek PHP Skala Enterprise (Clean Architecture)

Berikut adalah pola struktur folder profesional yang memisahkan tanggung jawab domain, infrastruktur, dan service layer:

```
sistem-akademik/
├── composer.json
├── vendor/                # Direktori pustaka & autoloader
├── autoload.php
├── src/                  # Kode aplikasi (App\)
```

```
    ├── Domain/
    │   ├── Model/        # App\Domain\Model\Mahasiswa
    │   └── Repository/   #
    └── App\Domain\Repository\MahasiswaRepoInterface
        └── Infrastructure/
```

```

|   |   | Persistence/           #
App\Infrastructure\Persistence\JsonMahasiswaRepo
|   |   | Service/              # App\Service\RegistrasiKrsService
|   |   | Helper/              # App\Helper\FormatRupiah
|   |   | tests/               # Unit Testing (Tests\)
|   |   | Domain/MahasiswaTest.php
|   |   | public/
|   |   | index.php            # Single Entry Point Aplikasi

```

```

<?php
// File: public/index.php
declare(strict_types=1);

// Cukup muat 1 baris autoloader Composer untuk seluruh aplikasi:
require_once __DIR__ . '/../vendor/autoload.php';

use App\Domain\Model\Mahasiswa;
use App\Service\KrsEngineService;

$mhs = new Mahasiswa("240101", "Cut Nyak Dhien", 3.92);
$engine = new KrsEngineService();
$maxSks = $engine->hitungMaksimalSks($mhs);

echo "Mahasiswa {$mhs->nama} memenuhi syarat mengambil maksimal {$maxSks} SKS.\n";

```



#### Tips Composer: Optimasi Produksi dengan Flag `--classmap-authoritative`

Saat melakukan deployment ke server produksi (Production CI/CD Pipeline), selalu jalankan 'composer install --no-dev --optimize-autoloader --classmap-authoritative'. Langkah ini menghilangkan overhead pemeriksaan filesystem stat() pada disk sehingga latency aplikasi PHP berkurang drastis.

## Rangkuman Materi

1. Namespace mengisolasi class, fungsi, dan konstanta guna mencegah tabrakan nama simbol (Name Collision).
2. Instruksi 'use' dan aliasing ('as') menyederhanakan penulisan Fully Qualified Name yang panjang.
3. Standar PSR-4 menetapkan pemetaan matematis antara Namespace dan struktur folder fisik proyek.
4. Composer mengotomatisasi autoloading class dan menyediakan fitur optimasi classmap untuk server produksi.
5. Pemisahan folder terstruktur (Domain, Infrastructure, Service) mewujudkan Clean Architecture yang maintainable.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan bahaya Global Namespace Pollution pada proyek PHP skala besar dan bagaimana Namespace menyelesaikannya!



2. Uraikan perbedaan antara Fully Qualified Name, Qualified Name, dan Unqualified Name dalam aturan resolusi simbol PHP!
3. Bagaimana cara kerja perintah ``composer dump-autoload --classmap-authoritative`` dalam meningkatkan performa server produksi?
4. Rancanglah konfigurasi ``composer.json`` untuk proyek yang memetakan namespace ``Universitas\Akademik`` ke folder ``src/`` dan namespace ``Tests`` ke folder ``tests/``!

## BAB 9

# PENANGANAN KESALAHAN (EXCEPTION HANDLING) & ROBUST ERROR FLOW

---

### Tujuan & Capaian Pembelajaran (Sub-CPMK 4)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami filosofi *Exception Handling* dan prinsip Fail-Fast dibandingkan metode *error* status code konvensional.
- Menguasai pohon hierarki antarmuka *Throwable*, serta membedakan secara tajam antara *Error*, *LogicException*, dan *RuntimeException*.
- Menerapkan blok kontrol *try-catch-finally*, teknik *Multi-Catch Exceptions*, dan penangkapan anonim (*Non-capturing Catches* di PHP 8.0+).
- Menggunakan fitur *throw as an Expression* (PHP 8.0+) pada *Null Coalescing* dan *ekspresi match*.
- Merancang Custom Domain Exceptions yang kaya konteks (Rich Contextual Exceptions) serta menerapkan *Exception Chaining / Wrapping* untuk melestarikan root cause stack trace.
- Mengonfigurasi penanganan kesalahan terpusat (Global Exception Handler) berstandar keamanan enterprise.

## 9.1 Filosofi Exception Handling & Hierarki `Throwable`

Dalam rekayasa perangkat lunak modern, penanganan kesalahan dengan mengembalikan kode status (seperti `return false` atau `return -1`) terbukti berbahaya karena rawan diabaikan oleh kode pemanggil, memicu fenomena Silent Failure di mana data korup tetap diproses lebih jauh.

Prinsip Fail-Fast menuntut program untuk segera menghentikan alur kerja normal dan melempar Exception terstruktur begitu terjadi anomali atau pelanggaran aturan bisnis. Sejak PHP 7+, seluruh galat disatukan di bawah payung antarmuka `Throwable`:

1. **\Error:** Mewakili kegagalan fatal pada mesin Zend Engine (misal `TypeError`, `DivisionByZeroError`, `ValueError`, `UnhandledMatchError`).
2. **\LogicException:** Mewakili bug logika yang seharusnya dapat dicegah saat penulisan kode (misal `InvalidArgumentException`, `DomainException`, `OutOfBoundsException`).
3. **\RuntimeException:** Mewakili kegagalan lingkungan runtime yang tidak dapat diprediksi saat kompilasi (misal `PDOException`, koneksi jaringan terputus, disk penuh).

## 9.2 Blok Kontrol `try-catch-finally` & Multi-Catch Syntax

Blok try-catch menangkap eksepsi yang dilempar, di mana blok catch harus diurutkan dari tipe yang paling spesifik ke tipe yang paling umum. Fitur Multi-Catch memungkinkan beberapa tipe eksepsi ditangani dalam satu blok yang sama menggunakan simbol pipe (|):

```
<?php
declare(strict_types=1);

namespace App\Keuangan;

use InvalidArgumentException;
use DomainException;
use Exception;
use Throwable;

function kalkulasiBagiHasil(float $modal, float $rasio): float {
    if ($modal <= 0) {
        throw new InvalidArgumentException("Modal investasi harus lebih besar dari 0!");
    }
    if ($rasio < 0.0 || $rasio > 1.0) {
        throw new DomainException("Rasio bagi hasil harus berada pada rentang 0.0 s.d. 1.0!");
    }
    return $modal * $rasio;
}

try {
    echo "Memulai kalkulasi investasi...\n";
    $hasil = kalkulasiBagiHasil(100_000_000.0, 0.25);
    echo "Hasil Bagi Hasil: Rp " . number_format($hasil, 0, ',', '.') . "\n";
} catch (InvalidArgumentException | DomainException $e) {
    // Multi-Catch Syntax
    echo "⚠️ [VALIDASI GAGAL] " . $e->getMessage() . "\n";
} catch (Exception $e) {
    echo "❌ [EXCEPTION UMUM] Terjadi kesalahan: " . $e->getMessage() . "\n";
} catch (Throwable $e) {
    echo "💣 [FATAL ENGINE ERROR] " . $e->getMessage() . "\n";
} finally {
    echo "🧹 [CLEANUP] Pelepasan sumber daya / penutupan koneksi selesai.\n";
}
```

## 9.3 Fitur Modern PHP 8+: `throw` Expression & Non-Capturing Catches

Sejak PHP 8.0, kata kunci throw bertindak sebagai ekspresi (expression), sehingga dapat dituliskan langsung di dalam operator Null Coalescing, Ternary, maupun Arrow Function:

```
<?php
declare(strict_types=1);

class ProfilPengguna
{
```

```

public function __construct(
    public readonly string $username,
    public readonly string $email
) {}

public static function dariArray(array $payload): self {
    return new self(
        // PHP 8.0: Throw Expression Langsung pada operator ??
        $payload['username'] ?? throw new \InvalidArgumentException("Username
wajib diisi!"),
        $payload['email'] ?? throw new \InvalidArgumentException("Email wajib
diisi!")
    );
}
}

```

## 9.4 Merancang Custom Domain Exceptions & Exception Chaining

Custom Domain Exception menyimpan data kontekstual yang kaya (Rich Contextual Exception) agar mempermudah penanganan di lapisan presentasi. Exception Chaining / Wrapping membungkus error level rendah (misal query database gagal) ke dalam service exception tingkat tinggi dengan tetap melestarikan root cause asli melalui parameter `$previous`:

```

<?php
declare(strict_types=1);

namespace App\Exception;

// Rich Contextual Custom Exception
class SaldoTidakCukupException extends \RuntimeException
{
    public function __construct(
        public readonly string $nomorRekening,
        public readonly float $saldoTersedia,
        public readonly float $nominalDiminta,
        int $code = 400,
        ?\Throwable $previous = null
    ) {
        $pesan = sprintf(
            "Penarikan gagal pada rekening [%s]. Saldo: Rp %s, Diminta: Rp %s.",
            $nomorRekening,
            number_format($saldoTersedia, 0, ',', '.'),
            number_format($nominalDiminta, 0, ',', '.')
        );
        parent::__construct($pesan, $code, $previous);
    }
}

```

## 9.5 Studi Kasus Transaksi Perbankan Terpadu

```

<?php
declare(strict_types=1);

```

```

namespace App\Domain;

class RekeningBank
{
    public function __construct(
        public readonly string $nomorRekening,
        private float $saldo
    ) {}

    public function getSaldo(): float { return $this->saldo; }

    public function tarik(float $nominal): void {
        if ($nominal <= 0) {
            throw new \InvalidArgumentException("Nominal penarikan harus bernilai positif.");
        }
        if ($nominal > $this->saldo) {
            throw new \App\Exception\SaldoTidakCukupException($this->nomorRekening, $this->saldo, $nominal);
        }
        $this->saldo -= $nominal;
    }
}

$akun = new RekeningBank("UUI-ACC-1024", 1_000_000.0);
try {
    $akun->tarik(1_500_000.0); // Memicu Exception
} catch (\App\Exception\SaldoTidakCukupException $e) {
    echo "❌ [TRANSAKSI DITOLAK] " . $e->getMessage() . "\n";
    echo "    Kekurangan Dana: Rp " . number_format($e->nominalDiminta - $e->saldoTersedia, 0, ',', '.') . "\n";
}

```

### 🎯 Tips Error Handling: Jangan Pernah Menelan Eksepsi Kosong (Empty Catch)

Hindari praktik buruk 'catch (Exception \$e) {}' tanpa aksi apa pun. Menelan eksepsi secara diam-diam akan menyembunyikan letak bug sistem. Selalu catat eksepsi ke berkas log terpusat (PSR-3 Logger) atau lemparkan kembali jika tidak dapat ditangani pada lapisan tersebut.

## Rangkuman Materi

1. Prinsip Fail-Fast menghentikan program seketika saat terjadi anomali untuk mencegah data korup.
2. Throwable memayungi Error (masalah mesin) dan Exception (kondisi yang dapat ditangani aplikasi).
3. Blok try-catch-finally mengelola alur eksepsi dan pelepasan sumber daya secara terjamin.
4. PHP 8.0+ mendukung Throw as an Expression dan Non-capturing Catches yang meringkas kode.
5. Exception Chaining (\$previous) melestarikan riwayat root cause error tanpa membocorkan detail internal ke pengguna.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan perbedaan mendasar antara `LogicException` dan `RuntimeException` dalam arsitektur aplikasi!
2. Bagaimana cara kerja fitur Multi-Catch Exception dalam menyederhanakan kode penanganan kesalahan?
3. Rancanglah sebuah Rich Custom Exception `BatasKreditTerlampauException` dengan properti `$limitKredit` dan `$pemakaianSaatIni`!
4. Mengapa Exception Chaining (`$previous`) sangat krusial saat melakukan debugging sistem pada server produksi?

## BAB 10

# KOLEKSI OBJEK (OBJECT COLLECTIONS) DAN MANIPULASI ARRAY MODERN

### Tujuan & Capaian Pembelajaran (Sub-CPMK 4)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami filosofi First-Class Collections (Object Calisthenics) dan batasan keamanan tipe (Type Safety) pada array bawaan PHP.
- Membangun Type-Safe Object Collection yang mengenkapsulasi array internal dan mencegah kontaminasi data heterogen.
- Mengintegrasikan antarmuka pustaka standar PHP (SPL): Countable, IteratorAggregate, ArrayAccess, dan JsonSerializable.
- Menguasai paradigma pemrograman fungsional pada manipulasi data menggunakan `array_map`, `array_filter`, `array_reduce`, dan Arrow Functions (`fn() =>`).
- Menerapkan fitur modern PHP 8+: Array Unpacking dengan String Keys (PHP 8.1+), `array_is_list()` (PHP 8.1+), serta Method Chaining (Fluent Interface).
- Merancang modul rekapitulasi data akademik skala besar berbasis Immutable Collection Transformation.

## 10.1 Filosofi First-Class Collections & Masalah Type Safety

Array bawaan PHP sangat fleksibel namun bersifat loosely-typed, di mana array dapat menampung campuran berbagai tipe data (integer, string, objek berbeda) dalam satu wadah tanpa adanya pengawasan kompilasi. Hal ini menimbulkan kerapuhan sistem ketika kode mengasumsikan seluruh elemen adalah instansi dari class tertentu.

Pola desain First-Class Collection (dicituskan oleh Jeff Bay dalam Object Calisthenics) mengajarkan bahwa setiap class yang memegang koleksi data tidak boleh memiliki variabel anggota lain selain koleksi itu sendiri. Pendekatan ini mengenkapsulasi array internal, menjamin Type Safety 100%, serta memusatkan logika analitik (seperti pencarian, penyaringan, dan agregasi) langsung pada objek koleksi.

## 10.2 Mengintegrasikan Antarmuka Standar PHP (SPL)

Agar objek koleksi dapat diperlakukan senyaman array asli PHP (dihitung dengan `count()`, diulang dengan `foreach`, dan deserialisasi dengan `json_encode()`), implementasikan antarmuka SPL berikut:

```
<?php
declare(strict_types=1);
```

```

namespace App\Domain\Model;

use Countable;
use IteratorAggregate;
use JsonSerializable;
use ArrayIterator;
use Traversable;

class Mahasiswa {
    public function __construct(
        public readonly string $nim,
        public string $nama,
        public float $ipk
    ) {}
}

class MahasiswaCollection implements Countable, IteratorAggregate, JsonSerializable
{
    /** @var Mahasiswa[] */
    private array $items = [];

    public function __construct(Mahasiswa ...$mahasiswa) {
        foreach ($mahasiswa as $mhs) {
            $this->tambah($mhs);
        }
    }

    public function tambah(Mahasiswa $mhs): self {
        $this->items[$mhs->nim] = $mhs;
        return $this;
    }

    public function count(): int {
        return count($this->items);
    }

    public function getIterator(): Traversable {
        return new ArrayIterator($this->items);
    }

    public function jsonSerialize(): array {
        return array_values($this->items);
    }
}

```

### 10.3 Manipulasi Fungsional: `map`, `filter`, `reduce`

Pendekatan pemrograman fungsional memastikan data asli tidak mengalami mutasi liar (Side-Effect Free) dengan selalu mengembalikan instance `MahasiswaCollection` baru saat dilakukan transformasi data:

```

<?php
// A. FILTERING: Mengembalikan MahasiswaCollection baru (Cum Laude)
public function filterCumLaude(): self {
    $hasil = array_filter($this->items, fn(Mahasiswa $m) => $m->ipk >= 3.50);
    $koleksiBaru = new self();
    $koleksiBaru->items = $hasil;
    return $koleksiBaru;
}

```



```

}

// B. REDUCING: Mengkalkulasi rata-rata IPK seluruh angkatan
public function hitungRataRataIpk(): float {
    if (empty($this->items)) return 0.0;
    $total = array_reduce($this->items, fn(float $sum, Mahasiswa $m) => $sum +
    $m->ipk, 0.0);
    return $total / count($this->items);
}

// C. SORTING: Mengurutkan IPK tertinggi ke terendah (Descending)
public function urutkanBerdasarkanIpkTertinggi(): self {
    $salinan = $this->items;
    uasort($salinan, fn(Mahasiswa $a, Mahasiswa $b) => $b->ipk <=> $a->ipk);
    $koleksiBaru = new self();
    $koleksiBaru->items = $salinan;
    return $koleksiBaru;
}

```

## 10.4 Studi Kasus Analisis Rekapitulasi Yudisium

```

<?php
declare(strict_types=1);

$angkatan2024 = new MahasiswaCollection(
    new Mahasiswa("240101", "Cut Meurah Intan", 3.92),
    new Mahasiswa("240102", "Teuku Rayhan", 3.45),
    new Mahasiswa("240103", "Siti Nurhaliza", 3.88),
    new Mahasiswa("240104", "Zulfa Safira", 3.95)
);

echo "Total Mahasiswa : " . count($angkatan2024) . " orang\n";
echo sprintf("Rata-rata IPK : %.2f\n", $angkatan2024->hitungRataRataIpk());

// Fluent Chaining: Filter CumLaude -> Urutkan Tertinggi
$cumlaudeSorted = $angkatan2024->filterCumLaude()->urutkanBerdasarkanIpkTertinggi();
foreach ($cumlaudeSorted as $mhs) {
    echo sprintf("🏆 [%s] %-20s : IPK %.2f\n", $mhs->nim, $mhs->nama, $mhs->ipk);
}

```

### 🎯 Tips Koleksi: Manfaatkan Immutable Methods pada Koleksi Objek

Rancang method manipulasi koleksi (seperti filter(), sort(), slice()) agar selalu mengembalikan instance baru ('return new self(...)') alih-alih mengubah array internal secara in-place. Pola ini menjaga integritas data asli dari modifikasi yang tidak diinginkan di bagian modul lain.

## Rangkuman Materi

1. First-Class Collections membungkus array di dalam class untuk menjamin type-safety 100%.
2. Antarmuka SPL (Countable, IteratorAggregate, JsonSerializable) membuat koleksi objek dapat diproses layaknya array native.
3. Manipulasi fungsional (map, filter, reduce) memproses data tanpa efek samping.

4. PHP 8.1+ mendukung Array Unpacking dengan String Keys dan fungsi validasi `array_is_list()`.

### Soal Latihan & Penugasan Mandiri

1. Jelaskan mengapa pendekatan First-Class Collection lebih unggul dibandingkan mengelola array mentah secara langsung!
2. Bagaimana cara kerja antarmuka `IteratorAggregate` dalam memungkinkan sebuah objek diulang menggunakan perulangan `foreach`?
3. Rancanglah method `ambilStokKritis(int $ambang)` pada `ProdukCollection` menggunakan fungsi `array_filter` dan Arrow Function!
4. Apa fungsi dari fungsi bawaan `array_is_list()` yang diperkenalkan pada PHP 8.1?

## BAB 11

# MANAJEMEN BERKAS DAN ALIRAN DATA (FILE HANDLING & I/O STREAM)

### Tujuan & Capaian Pembelajaran (Sub-CPMK 4)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami filosofi persistensi data berbasis berkas (File-based Persistence) serta perbedaan antara Stream-based I/O dan Buffered I/O.
- Menguasai teknik pencegahan Race Condition dan korupsi data menggunakan File Locking (flock) pada operasi konkuren.
- Menerapkan pustaka berorientasi objek modern SplFileObject dan SplFileInfo untuk membaca dan menulis berkas secara memory-efficient.
- Mengolah berkas berformat CSV dan JSON menggunakan standar keamanan modern (JSON\_THROW\_ON\_ERROR, json\_validate() di PHP 8.3+).
- Membangun pola desain File-Based Repository Pattern yang memisahkan logika persistensi berkas dari lapisan domain bisnis.
- Merancang modul Audit Logger persisten dengan rotasi berkas otomatis.

### 11.1 Filosofi Persistensi Berkas & Pencegahan Race Condition

Persistensi berkas merupakan fondasi paling mendasar dari penyimpanan data sebelum diperkenalkannya sistem basis data relasional. Pada aplikasi web yang melayani ratusan permintaan pengguna secara bersamaan (concurrent requests), operasi penulisan file rentan mengalami Race Condition yang menyebabkan isi berkas terpotong atau korup.

PHP menyediakan mekanisme penguncian berkas menggunakan fungsi flock(): LOCK\_SH (Shared Lock untuk operasi baca bersama), LOCK\_EX (Exclusive Lock untuk operasi tulis eksklusif), dan LOCK\_UN (Unlock untuk melepaskan kunci setelah operasi selesai).

### 11.2 Pemrosesan Berkas Berorientasi Objek dengan `SplFileObject`

Pustaka Standar PHP (SPL) menyediakan class SplFileObject yang membungkus fungsi I/O tradisional ke dalam antarmuka berorientasi objek yang hemat memori (Memory-Efficient Stream), sangat ideal untuk memproses berkas CSV besar baris demi baris:

```
<?php
declare(strict_types=1);
```

```

namespace App\Infrastructure\IO;

use SplFileObject;
use RuntimeException;

class CsvMahasiswaReader
{
    public function bacaData(string $pathFile): array {
        if (!file_exists($pathFile)) {
            throw new RuntimeException("Berkas CSV tidak ditemukan: {$pathFile}");
        }
        $file = new SplFileObject($pathFile, 'r');
        $file->setFlags(SplFileObject::READ_CSV | SplFileObject::SKIP_EMPTY |
SplFileObject::DROP_NEW_LINE);

        $daftarMahasiswa = [];
        $isHeader = true;

        foreach ($file as $baris) {
            if ($isHeader) { $isHeader = false; continue; }
            if (is_array($baris) && count($baris) >= 3) {
                $daftarMahasiswa[] = [
                    'nim' => trim($baris[0]),
                    'nama' => trim($baris[1]),
                    'ipk' => (float) trim($baris[2])
                ];
            }
        }
        return $daftarMahasiswa;
    }
}

```

### 11.3 Manipulasi JSON Modern & Validasi Native di PHP 8.3+

Pengolahan JSON modern diwajibkan menyertakan flag `JSON_THROW_ON_ERROR` untuk menangkap kesalahan parsing secara elegan. PHP 8.3 memperkenalkan fungsi native `json_validate()` untuk memverifikasi keabsahan sintaks JSON dengan performa tinggi tanpa alokasi memori berlebih:

```

<?php
declare(strict_types=1);

$dataAkademik = [
    'fakultas' => 'Fakultas Sains dan Teknologi',
    'prodi' => 'Informatika',
    'mahasiswa' => [
        ['nim' => '240101', 'nama' => 'Cut Meurah Intan', 'ipk' => 3.92],
        ['nim' => '240102', 'nama' => 'Teuku Rayhan', 'ipk' => 3.85]
    ]
];

// 1. Serialization dengan Error Throwing & Pretty Print
$jsonString = json_encode($dataAkademik, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE |
JSON_THROW_ON_ERROR);

// 2. Penulisan File Aman dengan Exclusive Lock (LOCK_EX)
file_put_contents('data_akademik.json', $jsonString, LOCK_EX);

```

```
// 3. PHP 8.3+: json_validate()
if (function_exists('json_validate') && json_validate($jsonString)) {
    echo "Format JSON valid dan terverifikasi secara native.\n";
}
```

## 11.4 Pola Desain: File-Based Repository Pattern

Pola Repository memisahkan lapisan proses bisnis dari detail media penyimpanan fisik. Berikut adalah implementasi JsonMahasiswaRepository lengkap dengan proteksi konkurensi:

```
<?php
declare(strict_types=1);

namespace App\Infrastructure\Persistence;

use App\Domain\Model\Mahasiswa;

class JsonMahasiswaRepository
{
    public function __construct(private string $filePath) {
        if (!file_exists($this->filePath)) {
            file_put_contents($this->filePath, json_encode([], JSON_PRETTY_PRINT),
LOCK_EX);
        }
    }

    public function ambilSemua(): array {
        $json = file_get_contents($this->filePath);
        $data = json_decode($json, true, 512, JSON_THROW_ON_ERROR);
        $koleksi = [];
        foreach ($data as $item) {
            $koleksi[$item['nim']] = new Mahasiswa($item['nim'], $item['nama'],
(float)$item['ipk']);
        }
        return $koleksi;
    }

    public function simpan(Mahasiswa $mhs): void {
        $semua = $this->ambilSemua();
        $semua[$mhs->nim] = $mhs;
        $arrayData = array_map(fn(Mahasiswa $m) => [
            'nim' => $m->nim,
            'nama' => $m->nama,
            'ipk' => $m->ipk
        ], array_values($semua));

        file_put_contents(
            $this->filePath,
            json_encode($arrayData, JSON_PRETTY_PRINT | JSON_THROW_ON_ERROR),
            LOCK_EX // Menjamin keamanan penulisan konkuren
        );
    }
}
```

 **Tips File I/O: Gunakan Stream I/O untuk Pemrosesan Berkas Skala Besar**

Untuk berkas berukuran besar (ratusan megabyte hingga gigabyte), hindari penggunaan `file_get_contents()` karena akan memuat seluruh isi berkas ke RAM sekaligus. Selalu gunakan `SplFileObject` atau `fgetcsv()` untuk memproses data secara bertahap (streaming).

## Rangkuman Materi

1. Persistensi berkas menyediakan media penyimpanan data mandiri tanpa ketergantungan DBMS luar.
2. File Locking (flock) dengan flag `LOCK_EX` wajib diterapkan untuk mencegah Race Condition saat penulisan.
3. `SplFileObject` menyediakan antarmuka OOP yang hemat memori untuk pemrosesan berkas teks dan CSV.
4. Flag `JSON_THROW_ON_ERROR` dan fungsi `json_validate()` (PHP 8.3+) meningkatkan keandalan parsing JSON.
5. Pola File-Based Repository memisahkan logika domain aplikasi dari sistem penyimpanan berkas.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan bahaya Race Condition pada penulisan berkas bersamaan dan bagaimana flag `LOCK_EX` mengatasinya!
2. Apa keunggulan menggunakan `SplFileObject` dibandingkan kombinasi fungsi prosedural `fopen` dan `fgets`?
3. Rancanglah class `DailyAuditLogger` yang mencatat log aktivitas dengan rotasi nama file harian (`audit_YYYY-MM-DD.log`)!
4. Bagaimana peran fungsi `json_validate()` pada PHP 8.3+ dalam mengoptimalkan performa API?

## BAB 12

# PRINSIP DESAIN PERANGKAT LUNAK SOLID PADA PHP MODERN

---

### Tujuan & Capaian Pembelajaran (Sub-CPMK 5)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami latar belakang lahirnya Prinsip Desain SOLID (Robert C. Martin / Uncle Bob) dalam mengeliminasi 4 gejala pembusukan arsitektur (Rigidity, Fragility, Immobility, Viscosity).
- Menerapkan Single Responsibility Principle (SRP) untuk mencegah anti pattern God Object dan memusatkan satu alasan perubahan per class.
- Menerapkan Open/Closed Principle (OCP) dengan pola Strategy Pattern sehingga sistem mudah diperluas tanpa mengubah kode yang sudah teruji.
- Menganalisis dan menegakkan Liskov Substitution Principle (LSP) termasuk aturan kovariansi (Covariance) dan kontravariansi (Contravariance) di PHP 8+.
- Menerapkan Interface Segregation Principle (ISP) dengan memecah Fat Interface menjadi Role-based Interfaces yang ramping.
- Menerapkan Dependency Inversion Principle (DIP) dan teknik Constructor Dependency Injection dengan fitur Constructor Property Promotion di PHP 8.0+.

## 12.1 Fondasi Teoritis SOLID: Menghindari Pembusukan Perangkat Lunak

Prinsip SOLID dicetuskan oleh Robert C. Martin (Uncle Bob) untuk memitigasi 4 gejala klasik pembusukan perangkat lunak (Software Rot): Rigidity (kekakuan sistem terhadap perubahan), Fragility (kerapuhan sistem di mana perubahan di satu modul merusak modul lain), Immobility (ketidakmampuan kode untuk digunakan kembali akibat tight coupling), dan Viscosity (sulitnya mempertahankan desain arsitektur yang benar).

## 12.2 Bedah Komprehensif 5 Prinsip SOLID di PHP 8+

1. Single Responsibility Principle (SRP): Setiap class hanya boleh memiliki satu alasan untuk berubah (satu tanggung jawab bisnis). Menghindari anti pattern God Object yang mencampurkan entitas, database, dan presentasi.
2. Open/Closed Principle (OCP): Entitas perangkat lunak harus terbuka untuk perluasan (open for extension), namun tertutup untuk modifikasi (closed for modification). Menggunakan abstraksi interface (seperti Strategy Pattern) agar fitur baru dapat disisipkan tanpa mengedit kode lama.

3. Liskov Substitution Principle (LSP): Objek turunan (subclass) harus dapat menggantikan objek induknya (superclass) tanpa merusak kebenaran program. Subclass dilarang memperketat prekondisi atau melempar eksepsi pada method yang sah di parent.
4. Interface Segregation Principle (ISP): Klien tidak boleh dipaksa bergantung pada antarmuka yang tidak digunakannya. Pecah antarmuka gemuk (Fat Interface) menjadi kumpulan Role Interfaces yang ramping.
5. Dependency Inversion Principle (DIP): Modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah; keduanya harus bergantung pada abstraksi interface.

```
<?php
declare(strict_types=1);

namespace App\Ecommerce;

// 1. OCP: Interface Strategi Diskon Terbuka
interface DiskonStrategyInterface {
    public function hitungDiskon(float $subtotal): float;
}

class DiskonMemberReguler implements DiskonStrategyInterface {
    public function hitungDiskon(float $subtotal): float { return $subtotal * 0.05; }
}

class DiskonFlashSale implements DiskonStrategyInterface {
    public function hitungDiskon(float $subtotal): float { return $subtotal * 0.20; }
}

// 2. DIP: Interface Payment Gateway Abstraksi
interface PaymentProcessorInterface {
    public function bayar(float $nominal): bool;
}

class QrisPaymentProcessor implements PaymentProcessorInterface {
    public function bayar(float $nominal): bool {
        echo "📄 Pembayaran QRIS Rp " . number_format($nominal, 0, ',', '.') . "
        BERHASIL.\n";
        return true;
    }
}

// 3. ISP: Interface Notifikasi Khusus
interface NotifierInterface {
    public function kirimNotifikasi(string $tujuan, string $pesan): bool;
}

class WhatsAppNotifier implements NotifierInterface {
    public function kirimNotifikasi(string $tujuan, string $pesan): bool {
        echo "📄 [WHATSAPP] Mengirim pesan ke {$tujuan}: {$pesan}\n";
        return true;
    }
}

// 4. SRP & DIP: Pipeline Checkout Tingkat Tinggi
class TransaksiPipelineService
{
    // PHP 8.0: Constructor Property Promotion dengan Type-Hint Abstraksi
```



```

public function __construct(
    private DiskonStrategyInterface $diskonStrategy,
    private PaymentProcessorInterface $paymentProcessor,
    private NotifierInterface $notifier
) {}

public function selesaikanTransaksi(string $customer, float $subtotal, string
$kontak): void {
    $diskon = $this->diskonStrategy->hitungDiskon($subtotal);
    $totalBayar = max(0.0, $subtotal - $diskon);

    echo "=====\n";
    echo "PROSES TRANSAKSI CHECKOUT E-COMMERCE (SOLID PIPELINE)\n";
    echo "Pelanggan   : {$customer}\n";
    echo "Subtotal    : Rp " . number_format($subtotal, 0, ',', '.') . "\n";
    echo "Diskon      : Rp " . number_format($diskon, 0, ',', '.') . "\n";
    echo "Total Bayar : Rp " . number_format($totalBayar, 0, ',', '.') . "\n";
    echo "-----\n";

    $this->paymentProcessor->bayar($totalBayar);
    $this->notifier-> kirimNotifikasi($kontak, "Pesanan Anda Rp " .
number_format($totalBayar) . " lunas.");
    echo "=====\n";
}
}

```

### Tips SOLID: Utamakan Constructor Dependency Injection

Hindari pembuatan instansi langsung ('new ClassKonkret()') di dalam service layer Anda. Selalu minta objek dependensi melalui parameter constructor dengan type-hint antarmuka (interface). Langkah ini menjamin kepatuhan mutlak terhadap DIP dan membuat kode Anda 100% siap untuk automated unit testing dengan Mockery atau PHPUnit.

## Rangkuman Materi

1. Prinsip SOLID mengeliminasi kerapuhan arsitektural (Software Rot) pada sistem perangkat lunak.
2. SRP memusatkan satu tanggung jawab per class untuk mencegah God Objects.
3. OCP memungkinkan perluasan fitur baru melalui Strategy Pattern tanpa modifikasi kode produksi.
4. LSP menjamin bahwa seluruh subclass dapat disubstitusikan secara aman menggantikan superclass.
5. ISP memecah interface gemuk menjadi role interfaces yang ramping dan tepat sasaran.
6. DIP dan Constructor Injection membalikkan arah ketergantungan modul ke abstraksi interface.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan 4 gejala utama Software Rot (Rigidity, Fragility, Immobility, Viscosity) dan bagaimana prinsip SOLID menyelesaikannya!

2. Bagaimana cara mengidentifikasi class yang melanggar Single Responsibility Principle (SRP)?
3. Berikan contoh kasus nyata pelanggaran Liskov Substitution Principle (LSP) pada hierarki bentuk geometri atau entitas perbankan!
4. Uraikan perbedaan konseptual antara Dependency Inversion Principle (prinsip), Dependency Injection (teknik), dan IoC Container (alat)!

## BAB 13

# ARSITEKTUR APLIKASI BERORIENTASI OBJEK (MODEL-SERVICE-REPOSITORY)

---

### Tujuan & Capaian Pembelajaran (Sub-CPMK 6)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Memahami filosofi Arsitektur Berlapis (Layered Architecture) dan Aturan Ketergantungan (Dependency Rule) dari Clean Architecture (Robert C. Martin, 2017).
- Membedakan peran dan tanggung jawab 4 lapisan konsentris: Entity, Use Case, Interface Adapter, dan Framework/Driver.
- Menguasai pola Model/Entity sebagai representasi data domain murni yang bebas dari dependensi infrastruktur.
- Menerapkan pola Repository sebagai abstraksi akses data yang memisahkan logika bisnis dari detail penyimpanan (Database Agnostic).
- Merancang Service Layer sebagai orkestrator aturan bisnis yang mengkoordinasikan Repository, Validator, dan Notifier.
- Membangun aplikasi CLI lengkap berpola Model-Service-Repository dengan persistensi JSON dan struktur folder terstandar PSR-4.

### 13.1 Filosofi Clean Architecture & Arsitektur Berlapis

Robert C. Martin dalam buku 'Clean Architecture: A Craftsman's Guide to Software Structure and Design' (2017) memperkenalkan arsitektur konsentris 4 lapisan dengan aturan ketergantungan (Dependency Rule) yang ketat: Dependensi kode sumber hanya boleh menunjuk ke arah dalam (dari lapisan luar ke lapisan dalam). Lapisan Entity dan Use Case tidak boleh mengetahui apapun tentang lapisan Framework/Driver (database, web framework, UI).

Tanpa pemisahan lapisan yang tegas, sebuah aplikasi monolitik akan mengalami masalah serius: logika bisnis tertanam di dalam controller (tidak dapat diuji tanpa server web), SQL query tersebar di berbagai class (perubahan skema memaksa modifikasi puluhan berkas), dan dependensi framework merembes ke inti domain (migrasi framework memaksa penulisan ulang total).

### 13.2 Lapisan 1: Entity / Domain Model

Entity merepresentasikan objek bisnis inti yang bebas dari dependensi infrastruktur (tidak mengimpor database, file system, atau framework). Menggunakan Backed Enum PHP 8.1+ untuk mendefinisikan state yang type-safe:

```

<?php
declare(strict_types=1);

namespace App\Domain\Model;

enum StatusPinjam: string {
    case Tersedia = 'tersedia';
    case Dipinjam = 'dipinjam';
    case Rusak    = 'rusak';
}

class Buku
{
    private StatusPinjam $status = StatusPinjam::Tersedia;

    public function __construct(
        public readonly string $isbn,
        public readonly string $judul,
        public readonly string $pengarang,
        public readonly int $tahunTerbit
    ) {}

    public function getStatus(): StatusPinjam { return $this->status; }

    public function pinjam(): void {
        if ($this->status !== StatusPinjam::Tersedia) {
            throw new \App\Domain\Exception\BukuTidakTersediaException(
                $this->isbn, $this->status
            );
        }
        $this->status = StatusPinjam::Dipinjam;
    }

    public function kembalikan(): void {
        $this->status = StatusPinjam::Tersedia;
    }
}

```

### 13.3 Repository Interface (Kontrak Abstraksi di Lapisan Domain)

Repository Interface dideklarasikan di lapisan domain (lapisan dalam), bukan di lapisan infrastruktur. Hal ini memastikan Service Layer (Use Case) hanya mengenal kontrak abstraksi dan tidak bergantung pada implementasi konkret penyimpanan data:

```

<?php
declare(strict_types=1);

namespace App\Domain\Repository;

use App\Domain\Model\Buku;

interface BukuRepositoryInterface
{
    public function simpan(Buku $buku): void;
    public function cariBerdasarkanIsbn(string $isbn): ?Buku;
    /** @return Buku[] */
    public function ambilSemua(): array;
}

```

```

    public function hapus(string $isbn): bool;
}

```

### 13.4 Service Layer (Use Case / Business Orchestrator)

Service Layer mengkoordinasikan Entity dan Repository tanpa mengetahui bagaimana data disimpan (JSON, database, atau API eksternal). Seluruh aturan bisnis dieksekusi pada lapisan ini:

```

<?php
declare(strict_types=1);

namespace App\Application\Service;

use App\Domain\Model\Buku;
use App\Domain\Repository\BukuRepositoryInterface;

class PerpustakaanService
{
    public function __construct(
        private BukuRepositoryInterface $bukuRepository
    ) {}

    public function daftarkanBukuBaru(string $isbn, string $judul, string $pengarang,
    int $tahun): Buku {
        if ($this->bukuRepository->cariBerdasarkanIsbn($isbn) !== null) {
            throw new \InvalidArgumentException("Buku [{$isbn}] sudah terdaftar!");
        }
        $buku = new Buku($isbn, $judul, $pengarang, $tahun);
        $this->bukuRepository->simpan($buku);
        return $buku;
    }

    public function pinjamBuku(string $isbn): Buku {
        $buku = $this->bukuRepository->cariBerdasarkanIsbn($isbn)
        ?? throw new \RuntimeException("Buku [{$isbn}] tidak ditemukan.");
        $buku->pinjam();
        $this->bukuRepository->simpan($buku);
        return $buku;
    }

    public function kembalikanBuku(string $isbn): Buku {
        $buku = $this->bukuRepository->cariBerdasarkanIsbn($isbn)
        ?? throw new \RuntimeException("Buku [{$isbn}] tidak ditemukan.");
        $buku->kembalikan();
        $this->bukuRepository->simpan($buku);
        return $buku;
    }
}

```

### 13.5 Implementasi Repository Konkret (Lapisan Infrastructure)

Implementasi konkret berada di lapisan luar (Interface Adapter) dan bertanggung jawab terhadap detail penyimpanan. Jika suatu hari Anda mengganti penyimpanan dari file JSON ke database MySQL (PDO), Anda hanya perlu membuat class

Hlm. 77

PdoBukuRepository implements BukuRepositoryInterface tanpa mengubah satu baris pun di folder Domain/ atau Application/:

```
<?php
declare(strict_types=1);

namespace App\Infrastructure\Persistence;

use App\Domain\Model\Buku;
use App\Domain\Model>StatusPinjam;
use App\Domain\Repository\BukuRepositoryInterface;

class JsonBukuRepository implements BukuRepositoryInterface
{
    public function __construct(private string $filePath) {
        if (!file_exists($this->filePath)) {
            file_put_contents($this->filePath, json_encode([], JSON_PRETTY_PRINT),
LOCK_EX);
        }
    }

    public function ambilSemua(): array {
        $json = file_get_contents($this->filePath);
        $data = json_decode($json, true, 512, JSON_THROW_ON_ERROR);
        return array_map(function (array $item): Buku {
            $buku = new Buku($item['isbn'], $item['judul'], $item['pengarang'],
$item['tahun_terbit']);
            if ($item['status'] === StatusPinjam::Dipinjam->value) { $buku->pinjam();
}
            return $buku;
        }, $data);
    }

    public function simpan(Buku $buku): void {
        $semua = $this->ambilSemua();
        $ditemukan = false;
        foreach ($semua as $i => $existing) {
            if ($existing->isbn === $buku->isbn) { $semua[$i] = $buku; $ditemukan =
true; break; }
        }
        if (!$ditemukan) { $semua[] = $buku; }
        $this->persistKeFile($semua);
    }

    public function cariBerdasarkanIsbn(string $isbn): ?Buku {
        foreach ($this->ambilSemua() as $buku) {
            if ($buku->isbn === $isbn) return $buku;
        }
        return null;
    }

    public function hapus(string $isbn): bool {
        $semua = $this->ambilSemua();
        $filtered = array_filter($semua, fn(Buku $b) => $b->isbn !== $isbn);
        if (count($filtered) === count($semua)) return false;
        $this->persistKeFile(array_values($filtered));
        return true;
    }

    private function persistKeFile(array $koleksi): void {
        $data = array_map(fn(Buku $b) => [
            'isbn' => $b->isbn, 'judul' => $b->judul,
```

```

        'pengarang' => $b->pengarang, 'tahun_terbit' => $b->tahunTerbit,
        'status' => $b->getStatus()->value
    ], $koleksi);
    file_put_contents($this->filePath, json_encode($data, JSON_PRETTY_PRINT |
JSON_THROW_ON_ERROR), LOCK_EX);
    }
}

```

### **Tips Arsitektur Bersih: Pisahkan Domain dari Infrastructure**

Selalu deklarasikan Repository Interface di folder Domain (lapisan dalam) dan implementasi konkritnya di folder Infrastructure (lapisan luar). Hal ini memastikan inti bisnis aplikasi Anda portable, tidak bergantung pada framework atau teknologi penyimpanan tertentu, serta siap 100% untuk automated unit testing menggunakan mock repository.

## Rangkuman Materi

1. Clean Architecture memisahkan aplikasi menjadi 4 lapisan konsentris dengan Dependency Rule mengarah ke dalam.
2. Entity/Domain Model merepresentasikan aturan bisnis murni yang bebas dari dependensi infrastruktur.
3. Repository Interface dideklarasikan di lapisan Domain sebagai kontrak abstraksi akses data.
4. Service Layer mengkoordinasikan Entity dan Repository tanpa mengetahui detail penyimpanan.
5. Implementasi konkrit (JsonRepository, PdoRepository) berada di lapisan Infrastructure yang dapat diganti tanpa memodifikasi inti bisnis.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan Aturan Ketergantungan (Dependency Rule) dalam Clean Architecture dan mengapa arah dependensi harus ke dalam!
2. Mengapa Repository Interface harus dideklarasikan di lapisan Domain dan bukan di lapisan Infrastructure?
3. Rancang implementasi kedua `PdoBukuRepository` menggunakan database SQLite tanpa mengubah `PerpustakaanService`!
4. Gambarkan diagram aliran ketergantungan (dependency flow) proyek perpustakaan ini dan pastikan tidak ada panah dari lapisan dalam ke lapisan luar!

## BAB 14

# STUDI KASUS MINI PROJECT: SISTEM POINT OF SALE (POS) TERPADU

### Tujuan & Capaian Pembelajaran (Sub-CPMK 6)

Setelah mempelajari dan menyelesaikan seluruh materi pada bab ini, mahasiswa diharapkan memiliki kompetensi untuk:

- Mengintegrasikan seluruh 14 bab materi: 4 Pilar OOP, Backed Enum, First-Class Collections, Rich Exceptions, File I/O Stream dengan Locking, dan Prinsip SOLID.
- Menerapkan arsitektur Model-Service-Repository (Clean Architecture) dengan pemisahan lapisan yang tegas dan kepatuhan mutlak pada Dependency Rule.
- Mengembangkan modul transaksi kasir retail berbasis CLI interaktif dengan kalkulasi multi-diskon (Strategy Pattern) dan multi-metode pembayaran.
- Menghasilkan kode program yang memenuhi standar PSR-4 Autoloading, PSR-12 Coding Style, serta aman dari Race Condition pada persistensi data.
- Mempresentasikan proyek perangkat lunak terintegrasi dengan dokumentasi teknis dan diagram UML yang lengkap.

### 14.1 Deskripsi Kasus & Kebutuhan Sistem POS Enterprise

Sistem Point of Sale (POS) Kasir Retail Terpadu merupakan proyek capstone yang mensintesis seluruh kompetensi pemrograman berorientasi objek yang telah dipelajari. Sistem ini bertanggung jawab mengelola master produk dengan kategori bertipe Backed Enum, memproses keranjang belanja pelanggan menggunakan First-Class Collection, menerapkan skema diskon fleksibel berbasis Strategy Pattern, memvalidasi pembayaran tunai dan non-tunai, serta mendokumentasikan struk transaksi secara persisten ke dalam berkas JSON yang terproteksi File Locking (LOCK\_EX).

### 14.2 Desain Domain Model & First-Class Collection

Lapisan domain dirancang menggunakan fitur-fitur modern PHP 8+: Backed Enum untuk kategori produk, readonly properties pada entitas, invariant validasi stok, dan First-Class Collection untuk keranjang belanja:

```
<?php
declare(strict_types=1);

namespace App\Domain\Model;

// 1. Backed Enum Kategori Produk
enum KategoriProduk: string {
    case Makanan = 'Makanan';
    case Minuman = 'Minuman';
    case Elektronik = 'Elektronik';
```



```

        case Pendidikan = 'Pendidikan';
    }

    // 2. Entity ItemProduk dengan Enkapsulasi Invariant
    class ItemProduk {
        public function __construct(
            public readonly string $sku,
            public readonly string $nama,
            public readonly float $harga,
            private int $stok,
            public readonly KategoriProduk $kategori
        ) {
            if ($harga <= 0) throw new \InvalidArgumentException("Harga harus positif!");
            if ($stok < 0) throw new \InvalidArgumentException("Stok tidak boleh
negatif!");
        }
        public function getStok(): int { return $this->stok; }
        public function kurangiStok(int $qty): void {
            if ($qty > $this->stok) {
                throw new \App\Domain\Exception\StokTidakCukupException($this->sku,
$this->stok, $qty);
            }
            $this->stok -= $qty;
        }
    }
}

    // 3. First-Class Collection Keranjang Belanja
    class KeranjangCollection implements \Countable, \IteratorAggregate {
        /** @var array<string, ItemKeranjang> */
        private array $items = [];
        public function tambah(ItemProduk $p, int $qty): void {
            $qtyTotal = ($this->items[$p->sku]->kuantitas ?? 0) + $qty;
            $this->items[$p->sku] = new ItemKeranjang($p, $qtyTotal);
        }
        public function hitungTotalBruto(): float {
            return array_reduce($this->items, fn($sum, $i) => $sum + $i->getSubtotal(),
0.0);
        }
        public function count(): int { return count($this->items); }
        public function getIterator(): \Traversable { return new
\ArrayIterator($this->items); }
        public function isEmpty(): bool { return empty($this->items); }
    }
}

```

### 14.3 Strategi Diskon & Pembayaran Polimorfik (SOLID OCP & DIP)

Kalkulasi diskon dan proses pembayaran diabstraksikan melalui interface sehingga mendukung penambahan varian strategi baru tanpa modifikasi kode yang sudah ada:

```

<?php
declare(strict_types=1);

namespace App\Domain\Diskon;

interface DiskonStrategyInterface {
    public function hitungDiskon(float $subtotal): float;
}

```

```

    public function getNama(): string;
}

class DiskonPersentase implements DiskonStrategyInterface {
    public function __construct(public readonly float $persen) {}
    public function hitungDiskon(float $subtotal): float { return $subtotal
* ($this->persen / 100); }
    public function getNama(): string { return "Diskon {$this->persen}%"; }
}

```

## 14.4 Application Service & Persistensi File-Based

KasirService mengorkestrasi alur transaksi, pemotongan stok otomatis, dan perekaman riwayat ke berkas JSON dengan proteksi LOCK\_EX:

```

<?php
declare(strict_types=1);

namespace App\Application\Service;

use App\Domain\Model\KeranjangCollection;
use App\Domain\Diskon\DiskonStrategyInterface;
use App\Domain\Pembayaran\PaymentProcessorInterface;
use App\Domain\Repository\ProdukRepositoryInterface;
use App\Domain\Repository\TransaksiRepositoryInterface;

class KasirService {
    public function __construct(
        private ProdukRepositoryInterface $produkRepo,
        private TransaksiRepositoryInterface $transaksiRepo
    ) {}

    public function checkout(
        KeranjangCollection $keranjang,
        DiskonStrategyInterface $diskon,
        PaymentProcessorInterface $payment,
        float $nominalBayar
    ): array {
        if ($keranjang->isEmpty()) throw new \RuntimeException("Keranjang belanja
kosong!");
        $subtotal = $keranjang->hitungTotalBruto();
        $potongan = $diskon->hitungDiskon($subtotal);
        $totalAkhir = max(0.0, $subtotal - $potongan);

        $payment->prosesBayar($totalAkhir, $nominalBayar);

        foreach ($keranjang as $item) {
            $item->produk->kurangiStok($item->kuantitas);
            $this->produkRepo->simpan($item->produk);
        }

        $dataTrx = [
            'id' => 'TRX-' . date('YmdHis'),
            'waktu' => date('Y-m-d H:i:s'),
            'subtotal' => $subtotal,
            'diskon' => $potongan,
            'total' => $totalAkhir,

```

```

        'metode' => $payment->getMetode()
    ];
    $this->transaksiRepo->simpan($dataTrx);
    return $dataTrx;
}
}

```

### 🎯 Tips Evaluasi Proyek Capstone: Pastikan Kualitas Kode Terstandar PSR-12

Sebelum mengumpulkan proyek capstone, lakukan analisis statis menggunakan PHPStan (level 5+) dan periksa kepatuhan gaya penulisan kode menggunakan PHP\_CodeSniffer (PSR-12). Pastikan seluruh unit test lolos 100% dan berkas database JSON terlindungi dengan penguncian LOCK\_EX.

## Rangkuman Materi

1. Proyek POS memadukan seluruh 14 bab materi: pilar OOP, collections, exceptions, I/O, dan SOLID.
2. Model-Service-Repository menjamin kemandirian proses bisnis dari media penyimpanan.
3. Strategy Pattern mempermudah ekspansi jenis diskon dan metode pembayaran baru.
4. Penguncian berkas LOCK\_EX wajib diterapkan untuk menjamin integritas data transaksi kasir.

## Soal Latihan & Penugasan Mandiri

1. Jelaskan bagaimana prinsip Dependency Inversion (DIP) diterapkan pada `KasirService` dan keuntungan apa yang diperoleh saat pengujian otomatis!
2. Bagaimana mekanisme penguncian berkas `LOCK\_EX` mencegah kehilangan data (\*Lost Update\*) saat dua kasir melakukan checkout bersamaan?
3. Rancanglah strategi diskon baru `DiskonBertingkat` (misal diskon 15% jika belanja di atas Rp 500.000) tanpa mengubah `KasirService`!
4. Uraikan langkah-langkah implementasi `PdoTransaksiRepository` untuk menggantikan repository JSON saat proyek beralih ke MySQL!

## GLOSARIUM ISTILAH OOP & PHP

| Istilah               | Penjelasan / Definisi Teknis                                                                      |
|-----------------------|---------------------------------------------------------------------------------------------------|
| Abstract Class        | Class induk yang tidak dapat diinstansiasi langsung dan memuat definisi method abstrak.           |
| Autoloading           | Mekanisme pemuatan berkas class PHP secara otomatis pada saat class tersebut dipanggil.           |
| Backed Enum           | Tipe data enumerasi native PHP 8.1 yang nilainya terikat pada string atau integer skalar.         |
| Class                 | Cetak biru (*blueprint*) yang mendefinisikan atribut dan perilaku dari suatu tipe data objek.     |
| Constructor Promotion | Fitur PHP 8.0 yang menggabungkan deklarasi properti dan penugasan pada parameter constructor.     |
| Dependency Injection  | Teknik memasukkan objek dependensi dari luar melalui constructor/setter.                          |
| Dynamic Dispatch      | Mekanisme runtime PHP dalam menentukan method mana yang dieksekusi berdasarkan tipe objek aktual. |
| Encapsulation         | Pilar pembungkusan data dan method serta pembatasan akses langsung dari luar class.               |
| Inheritance           | Pewarisan atribut dan method dari superclass ke subclass menggunakan kata kunci `extends`.        |
| Interface             | Kontrak murni tanpa implementasi yang mendikte method apa saja yang wajib disediakan oleh class.  |
| Named Arguments       | Pemanggilan fungsi/method dengan menyebutkan nama parameternya secara eksplisit (PHP 8.0+).       |
| Polymorphism          | Prinsip satu antarmuka yang merespons dengan banyak implementasi perilaku yang berbeda.           |
| PSR-4                 | Standar rekomendasi PHP-FIG mengenai pemetaan otomatis antara namespace dan direktori berkas.     |

|                   |                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------|
| Readonly Property | Properti yang nilainya hanya dapat diinisialisasi satu kali dan bersifat immutable (PHP 8.1+).      |
| SOLID             | Lima prinsip dasar rekayasa perangkat lunak berorientasi objek untuk arsitektur yang bersih.        |
| Trait             | Mekanisme horizontal code reuse untuk berbagi kode lintas class di luar hierarki pewarisan tunggal. |

## DAFTAR PUSTAKA

- Bay, J. (2008). Object Calisthenics. In *The ThoughtWorks Anthology: Essays on Software Technology and Innovation* (pp. 83–98). Pragmatic Bookshelf.
- Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley Professional.
- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
- Cardelli, L., & Wegner, P. (1985). On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys (CSUR)*, 17(4), 471–523. <https://doi.org/10.1145/6041.6042>
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Liskov, B. (1987). Data abstraction and hierarchy. *ACM SIGPLAN Notices*, 23(5), 17–34. <https://doi.org/10.1145/62139.62141>
- Lockhart, J. (2015). *Modern PHP: New Features and Good Practices*. O'Reilly Media.
- MacArthur, K. (2022). *PHP 8 Quick Scripting Reference* (3rd ed.). Apress. <https://doi.org/10.1007/978-1-4842-8111-6>
- Martin, R. C. (2002). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.
- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Meyer, B. (1988). *Object-Oriented Software Construction*. Prentice Hall.
- Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>
- PHP Documentation Group. (2025). *PHP Manual: Classes and Objects (PHP 8.0 – 8.4)*. <https://www.php.net/manual/en/language.oop5.php>
- PHP Documentation Group. (2025). *PHP Manual: Predefined Exceptions and Standard PHP Library (SPL)*. <https://www.php.net/manual/en/spl.exceptions.php>
- PHP Foundation. (2024). *PHP 8.4 Release Announcement and Core Language Enhancements*. <https://www.php.net/releases/8.4/en.php>
- PHP-FIG. (2012). *PSR-3: Logger Interface Specification*. <https://www.php-fig.org/psr/psr-3/>
- PHP-FIG. (2014). *PSR-4: Autoloader Standard Specification*. <https://www.php-fig.org/psr/psr-4/>
- PHP-FIG. (2019). *PSR-12: Extended Coding Style Guide*. <https://www.php-fig.org/psr/psr-12/>
- Saradhna, M. (2021). *Mastering PHP 8: Build enterprise-ready applications using the new features, architecture, and design patterns*. Packt Publishing.

- Seemann, M., & Steven, M. (2019). *Dependency Injection Principles, Practices, and Patterns*. Manning Publications.
- Skvorc, B. (2017). *PHP Application Development with Composer, PHP-FIG, and More*. SitePoint.
- Zandstra, M. (2021). *PHP 8 Objects, Patterns, and Practice: Mastering OO Enhancements, Design Patterns, and Test-Driven Development* (6th ed.). Apress.  
<https://doi.org/10.1007/978-1-4842-6791-2>

## PROFIL TIM PENULIS

### 1. Mahendar Dwi Payana, S.ST., M.T.

Dosen tetap pada Program Studi Informatika, Fakultas Sains dan Teknologi, Universitas Ubudiyah Indonesia (UUI), Banda Aceh. Beliau aktif mengampu dan meneliti dalam rumpun keilmuan Rekayasa Perangkat Lunak, Pemrograman Berorientasi Objek, Pemrograman Web, Pemrograman Mobile, serta Algoritma dan Struktur Data. Berkomitmen kuat pada pengembangan kurikulum OBE dan integrasi standar industri *software engineering*.

### 2. Desita Ria Yusian TB, S.ST., M.T

Dosen tetap pada Program Studi Informatika, Fakultas Sains dan Teknologi, Universitas Ubudiyah Indonesia (UUI), Banda Aceh. Beliau memiliki kepakaran dalam bidang Rekayasa Perangkat Lunak, Manajemen Basis Data, dan Analisis Perancangan Sistem Informasi.

### 3. M.Bayu Wibawa, S.Kom., MMSI

Dosen tetap pada Program Studi Informatika, Fakultas Sains dan Teknologi, Universitas Ubudiyah Indonesia (UUI), Banda Aceh. Beliau aktif dalam pengajaran dan penelitian di bidang Sistem Informasi Enterprise, Tata Kelola Teknologi Informasi, serta Rekayasa Perangkat Lunak Modern.

### 4. Nurul Hamdi, S.T., M.Kom

Dosen tetap pada Program Studi Informatika, Fakultas Sains dan Teknologi, Universitas Ubudiyah Indonesia (UUI), Banda Aceh. Beliau berfokus pada bidang Arsitektur Jaringan Komputer, Keamanan Sistem, dan Pengembangan Aplikasi Terdistribusi Berorientasi Objek.

### 5. Zuhar Musliyana, S.ST., M.T

Dosen tetap pada Program Studi Informatika, Fakultas Sains dan Teknologi, Universitas Ubudiyah Indonesia (UUI), Banda Aceh. Beliau memiliki fokus keahlian pada bidang Kecerdasan Buatan (Artificial Intelligence), Data Science, dan Implementasi Perangkat Lunak Terapan.